NEURAL NETWORKS / DAY 2

Training neural networks reliably

Diagnostics, initialization, normalization, optimization, regularization, and framework practice

Full day · 09:00–17:00

5 parts · 2 laboratories

PyTorch core · Keras alternative

Today's plan

09:00	Welcome, then Part 1: diagnose before you tune	Lecture
09:50	Part 2: signal propagation and normalization	Lecture
10:35	Break	Break
10:50	Part 3: a PyTorch training loop you can trust	Lecture
11:30	Lab 1: a reproducible PyTorch pipeline	Lab
12:30	Lunch	Lunch
13:30	Lab 1 debrief, then Part 4: optimizers, schedules, and tuning	Lecture
14:35	Break	Break
14:50	Part 5: regularization and autoencoders	Lecture
15:35	Lab 2, debrief, and exit check	Lab

LEARNING OUTCOMES

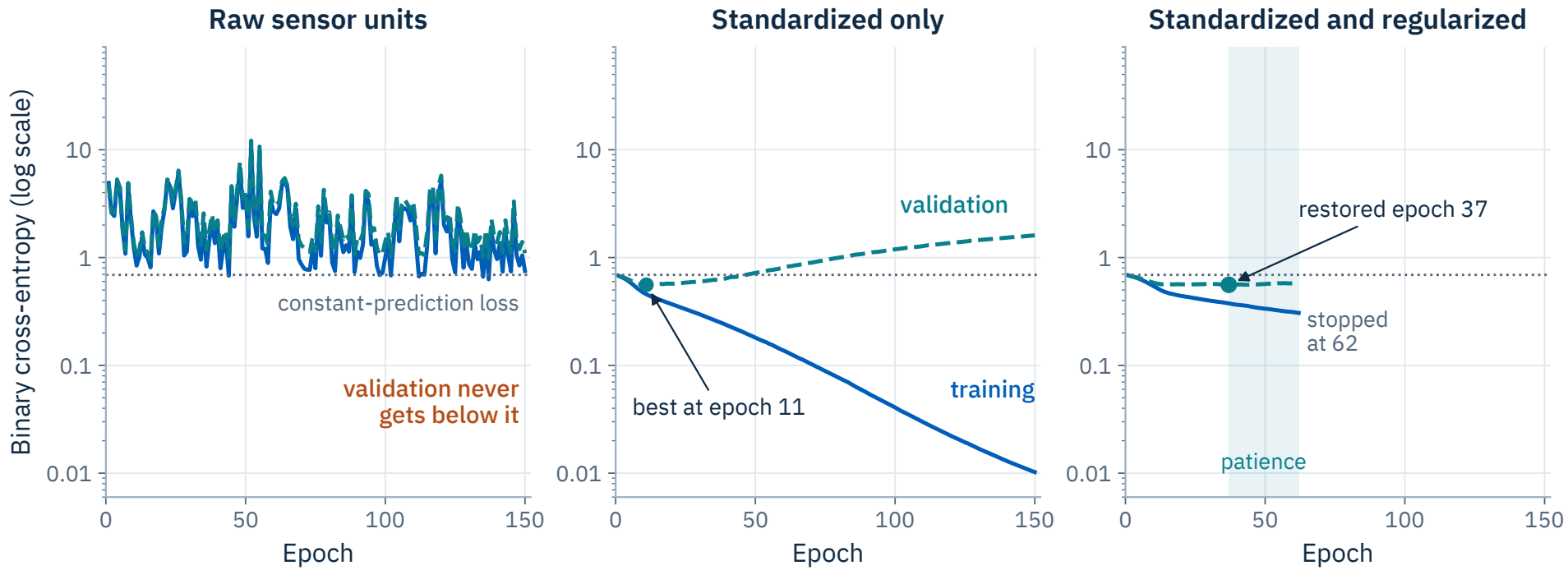
What you will be able to do

By the end of the day you will be able to:

1. Run a fixed diagnostic sequence: initial loss, one-batch fit, gradient flow, learning curves.
2. Explain how initialization, normalization, and skip connections keep signals at a trainable scale.
3. Write a PyTorch loop with correct train and eval modes, a copied checkpoint, and a reload check.
4. Choose an optimizer, learning rate, schedule, and batch size from evidence, not habit.
5. Match weight decay, dropout, early stopping, and augmentation to the symptom they treat.
6. Explain how an autoencoder turns reconstruction error into an anomaly score, and its limits.

Same network, same data, three different outcomes

A 12-input MLP learns a pump-failure flag from a synthetic ESP table with 240 training rows. Only the training setup changes.



Computed: synthetic ESP-style data, seeds 1 and 2 (raw pressures in kPa next to vibration in g), MLP 12-64-64-1, AdamW, torch seed 0; the dotted line is the loss of always predicting the training base rate.

PRACTICE

5 min · Pairs

Warm-up: which run would you trust, and why?

Look at the three runs on the previous slide.

1. Which run would you hand to an operations team, and what evidence would you ask for first?
2. The middle run reaches a training loss of 0.01. What does that number prove?
3. Name the first setting you would check in the left-hand run.

PART 1 OF 5

01

Diagnose before you tune

Most failed runs are pipeline bugs, not architecture problems.

- A fixed order of checks
- Initial loss and one-batch memorization
- Gradient flow, layer by layer
- Reading learning curves

A diagnostic sequence saves time

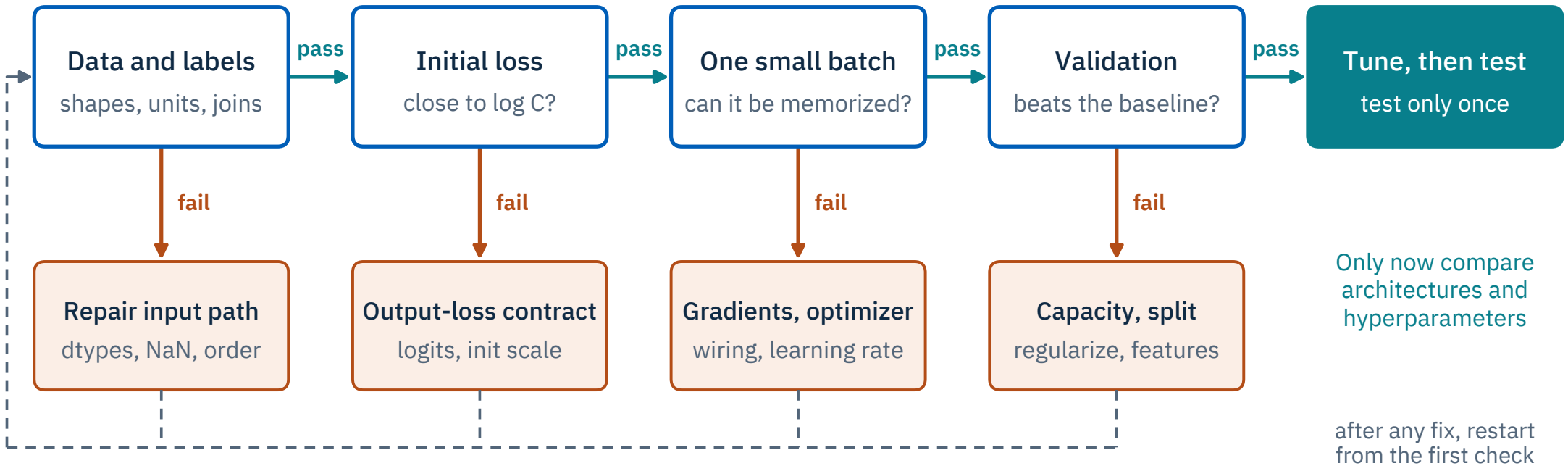
Check	Expected evidence	If it fails
Data and labels	Correct shapes, units, and alignment	Repair the input path
One forward pass	Finite outputs and plausible loss	Inspect output-loss contract
Tiny batch fit	Training loss falls substantially	Inspect gradients and optimizer
Validation fit	Generalization beyond fitted rows	Review capacity, split, and regularization

Run the checks in this order; each one assumes the previous one passed.

◆ A failure early in the list makes every later result meaningless.

Debug in a fixed order: data, contract, fit, generalize

Each check is cheap; skipping one usually costs hours later.

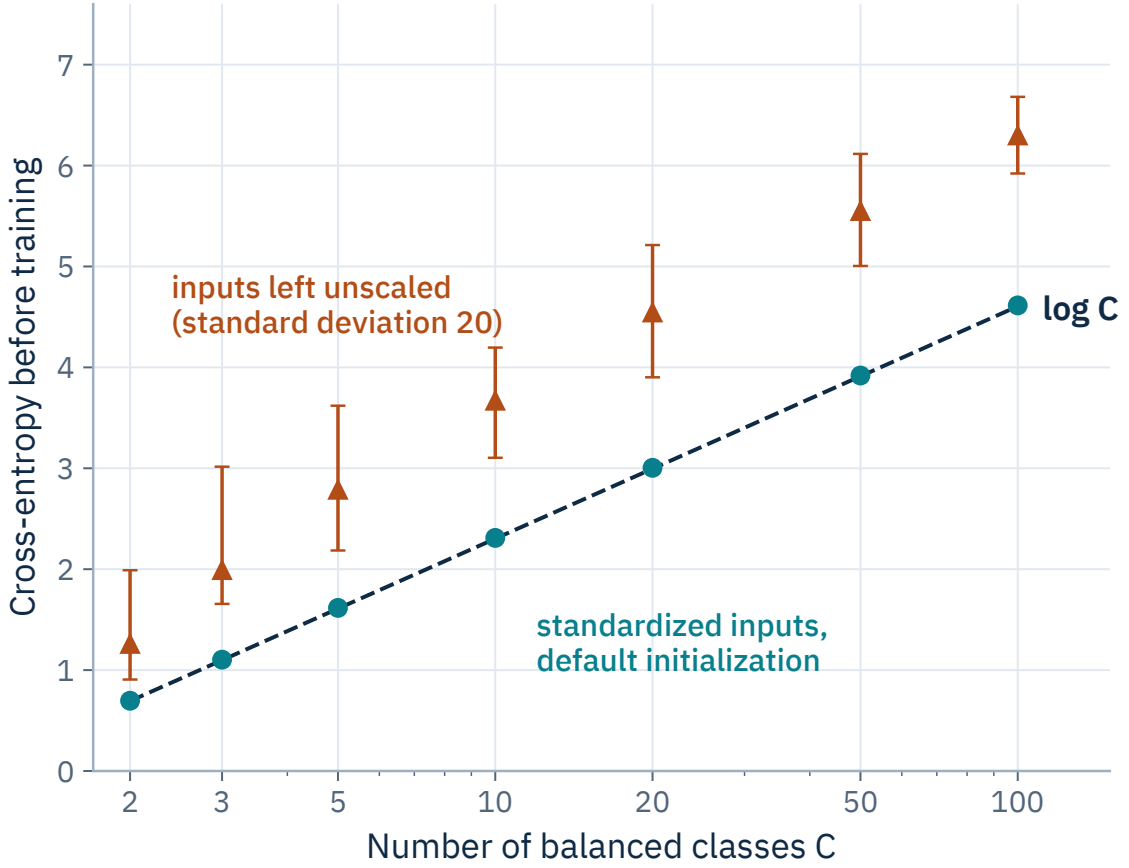


Schematic. A failed check sends you to its repair box; after any fix, restart from the first check.

Initial loss should be close to log C

$$L_0 = -\log \frac{1}{C} = \log C$$

- Untrained logits are near zero: predictions near uniform
- Binary: log 2 = 0.693; ten classes: log 10 = 2.303
- Far higher: logits too large, or a wrong loss contract
- Imbalanced classes: use the entropy of the class prior



Computed: 20 seeds per point, PyTorch default initialization; the orange points use the same inputs multiplied by 20.

Put the sanity checks in code, not in memory

```python
import math

xb, yb = next(iter(train_loader))
assert xb.shape == (16, 60) and yb.shape == (16, 1)
assert torch.isfinite(xb).all()
assert set(yb.unique().tolist()) <= {0.0, 1.0}

model.eval()
with torch.no_grad():
    loss0 = criterion(model(xb), yb).item()
print(f"initial loss {loss0:.3f}   log 2 = {math.log(2):.3f}")
```

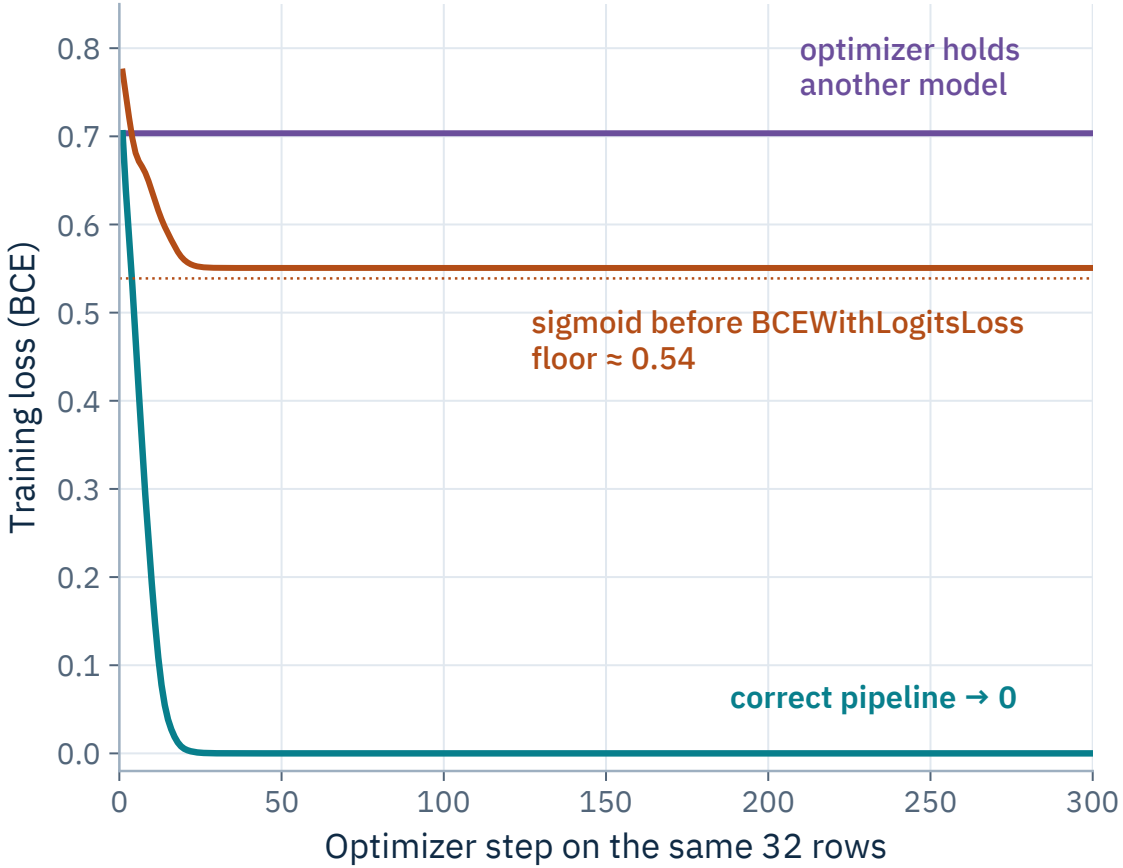
OUTPUT

```
initial loss 0.703   log 2 = 0.693
```

Lab 1 data and untrained model (seed 42); one batch of 16 rows varies by a few hundredths.

A healthy pipeline can memorize one small batch

- Train on the same 32 rows for a few hundred steps
- Switch off dropout and weight decay for this test
- Loss near zero: forward, loss, backward, and step are wired correctly
- A floor or a flat line points to a contract or optimizer bug



◆ Memorizing one batch proves the plumbing, not generalization.

Computed: 32 rows of the synthetic ESP table (seed 1), MLP 12-64-64-1, Adam lr 0.01, torch seed 0; both bugs injected on purpose.

Check that every layer receives a gradient

```
loss = criterion(model(xb), yb)
loss.backward()
for name, p in model.named_parameters():
    g = "None" if p.grad is None else f"{p.grad.norm():.4f}"
    print(f"{name:11s} {str(tuple(p.shape)):9s} {g}")
```

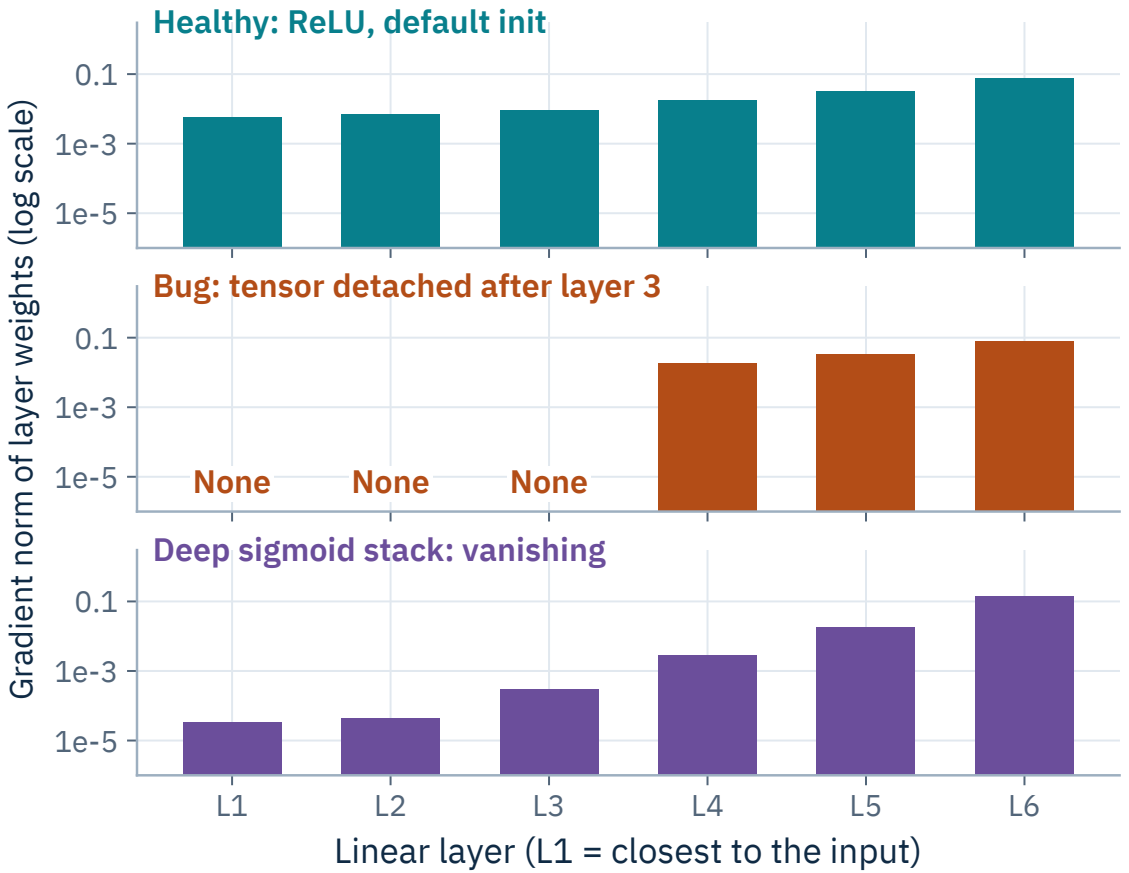
OUTPUT

```
fc1.weight  (32, 60)  0.5140
fc1.bias    (32,)     0.0632
fc2.weight  (32, 32)  None
fc2.bias    (32,)     None
out.weight  (1, 32)   0.3493
out.bias    (1,)      0.1227
```

Here `fc2` is defined in `__init__` but never called in `forward()`, so it can never learn.

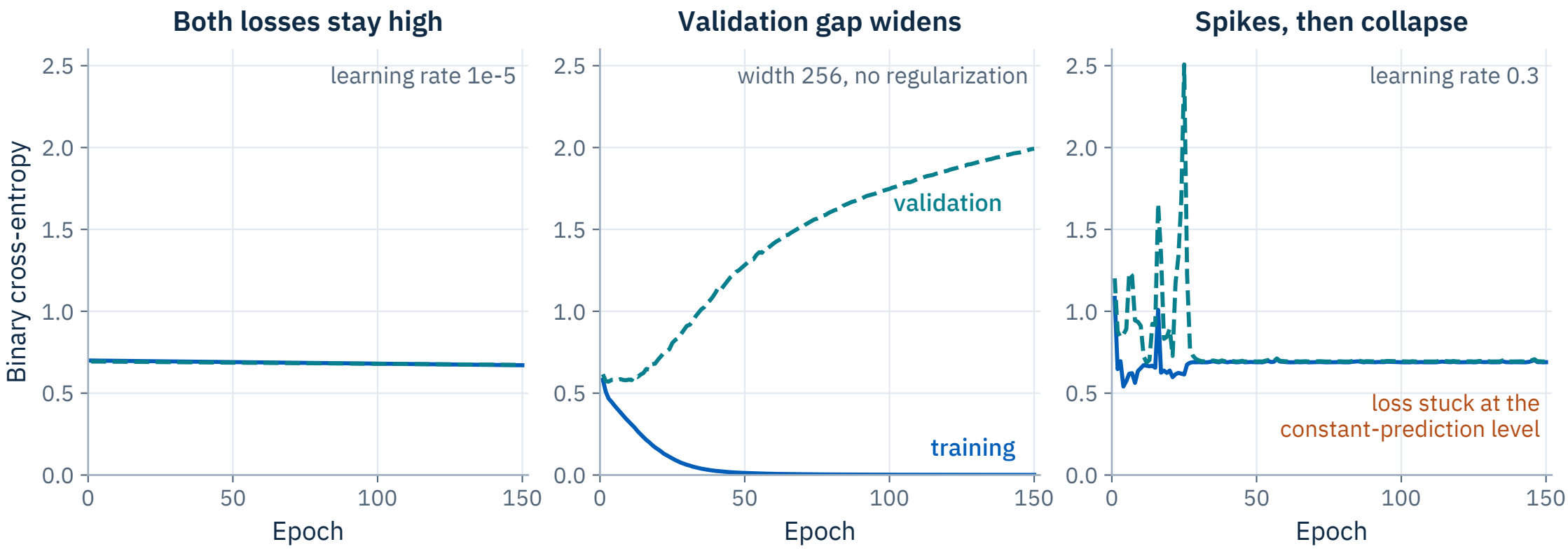
Missing or zero gradients point to a cut or dead path

- Compare gradient norms layer by layer after one backward pass
- None: detached, frozen, or unused; exactly zero: dead units
- Tiny near the input, large near the output: vanishing gradients
- Log these norms during training, not only when debugging



Computed: 6-layer MLPs of width 32, one backward pass on 256 random rows, torch seed 0; "None" marks layers that received no gradient.

Learning curves distinguish several problems



Computed: synthetic ESP table (seeds 1 and 2), two hidden layers, AdamW, torch seed 0; one setting changed per panel. Similar shapes can have other causes, so treat them as leads, not diagnoses.

DIAGNOSE BEFORE YOU TUNE

Read the curves before changing the model

Both losses high

- Check learning rate and input scaling first
- Then capacity and feature content

Validation gap widens

- Regularize or stop earlier
- Check the split for distribution shift

Spikes or NaN

- Lower the learning rate; clip gradients
- Look for bad rows, log(0), division by zero

Validation below training

- Dropout is active in the training loss
- Easier or leaked validation rows?

◆ Change one thing, rerun, and compare on the same validation rows.

PRACTICE

6 min · Pairs

Debugging from observations

Training loss remains flat near 0.69. Predictions are almost constant.

1. List two checks you would run before adding more layers.
2. What experiment would separate a data bug from an optimization problem?
3. The initial loss was 0.69 in one run and 7.5 in another. What does each suggest?

DISCUSSION

Check the path before the architecture

Data and labels

- Targets still aligned after joins and sorts
- Float targets of shape [batch, 1]

Output-loss contract

- Raw logits into BCEWithLogitsLoss
- No sigmoid inside the model

Gradients and optimizer

- Optimizer built from this model's parameters
- zero_grad, backward, and step all run

Only then capacity

- Overfit one batch first
- Then widen or deepen in controlled steps

PART 2 OF 5

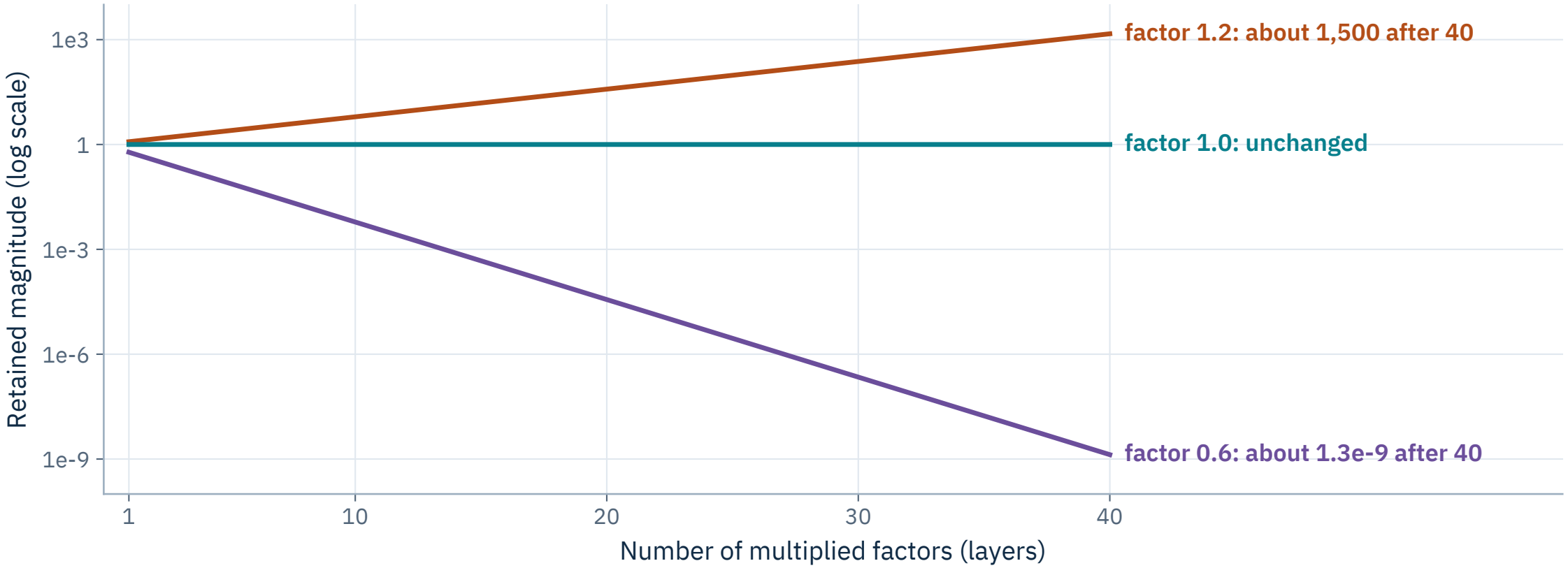
02

Signal propagation and normalization

Deep networks train when signals keep a stable scale.

- Why signals vanish or explode
- Initialization matched to the activation
- Batch, layer, and group normalization
- Residual connections as a gradient highway

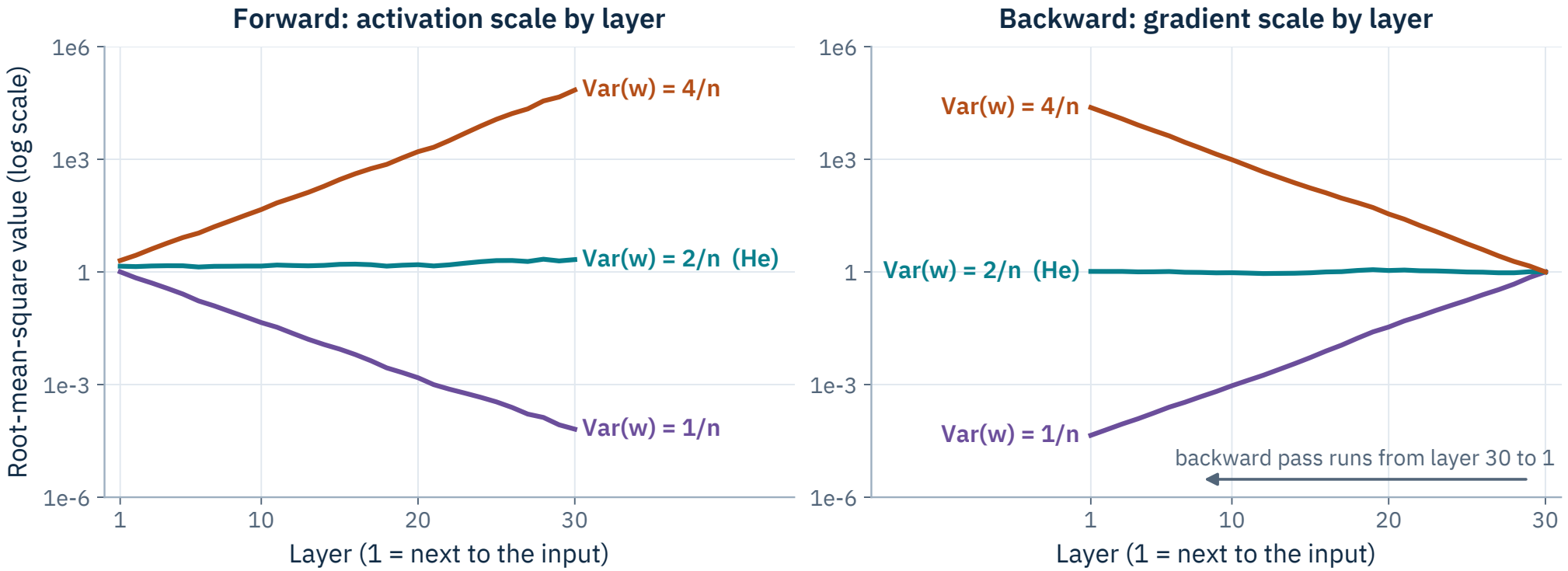
Gradient products can shrink or explode



Computed: analytical products of a repeated per-layer factor. The curves illustrate a mechanism, not measured gradients from a trained network.

Weight scale decides whether signals vanish or explode

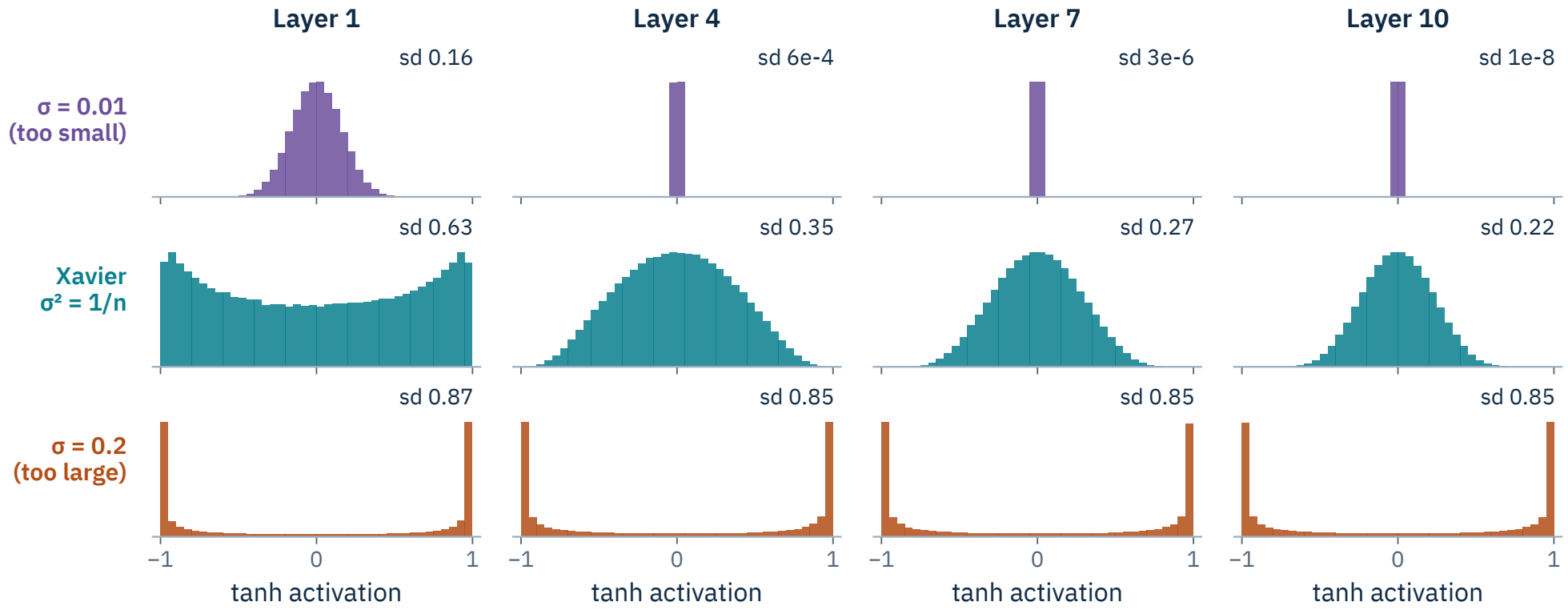
Each ReLU layer multiplies the signal's variance by about $n \cdot \mathrm{Var}(w)/2$; over 30 layers small differences compound.



Computed: 30-layer ReLU network, width n = 256, Gaussian weights, no biases or normalization; numpy seed 0.

Activation histograms reveal a bad initialization

A 10-layer tanh network: too small collapses to zero, too large saturates at ±1.



Computed: tanh network of width 256, Gaussian weights with the stated standard deviation, no biases, numpy seed 0; histograms of all activations at four depths.

Record activation statistics with forward hooks

```
stats = {}
def record(name):
    def hook(module, inputs, output):
        stats[name] = output.std().item()
    return hook

for name, m in model.named_modules():
    if isinstance(m, nn.Tanh):
        m.register_forward_hook(record(name))
with torch.no_grad():
    model(xb)
print({k: f"{v:.1e}" for k, v in stats.items()})
```

OUTPUT

```
{'1': '1.6e-01', '3': '2.5e-02', '5': '4.0e-03', '7': '6.4e-04'}
```

Four tanh layers of width 256, weights drawn with standard deviation 0.01, zero biases: the spread shrinks about 6× per layer.

Initialization controls the first signal scale

✗ Identical weights

- Every hidden unit computes the same function
- Gradients stay identical, so symmetry never breaks

Too small

- Activations and gradients shrink layer by layer
- Deep stacks stop learning

Too large

- ReLU signals explode; tanh and sigmoid saturate
- Loss spikes or NaN from the first steps

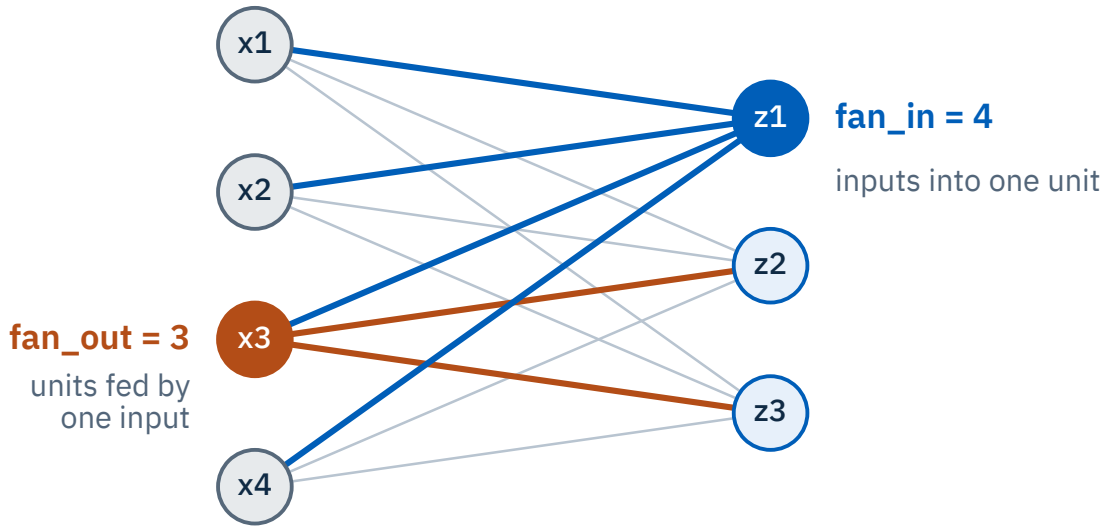
✓ Variance-preserving

- Xavier for tanh-like units, He (Kaiming) for ReLU
- Check both forward and backward scale

Initialization variance scales with fan-in

$$\mathrm{Var}(w) \approx \frac{2}{\mathrm{fan}_{\mathrm{in}} + \mathrm{fan}_{\mathrm{out}}} \quad \text{or} \quad \frac{2}{\mathrm{fan}_{\mathrm{in}}}$$

- Xavier (Glorot) averages fan-in and fan-out: suits tanh and sigmoid
- He (Kaiming) uses 2 / fan-in: compensates for ReLU zeroing half its inputs
- Distribution (normal or uniform) and gain change the exact bounds



$$\mathrm{Var}(z) = \mathrm{fan}_{\mathrm{in}} \cdot \mathrm{Var}(w) \cdot \mathrm{E}[x^2]$$

keep Var(z) steady → Var(w) ≈ 1 / fan_in (× 2 for ReLU)

Schematic: one dense layer with fan-in 4 and fan-out 3.

PRACTICE

5 min · Individual

How fast does a bad scale compound?

A 10-layer ReLU network has fan-in 256 in every layer; weights are Gaussian with standard deviation σ.

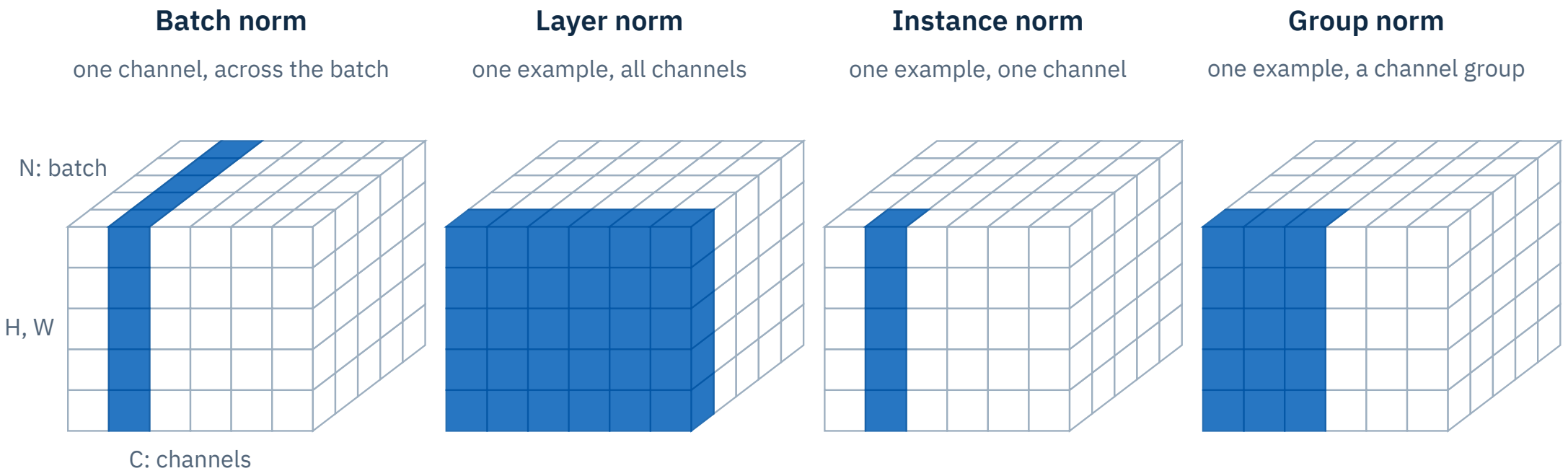
1. Compute the per-layer variance factor $256\,\sigma^2/2$ for $\sigma = 0.01$, $\sigma = 0.1$, and He initialization.
2. By what factor has the activation standard deviation changed after 10 layers in each case?
3. Which σ does He initialization give here?

Normalization uses different axes

Method	Statistics used	Training and inference issue
Input standardization	Training-set feature statistics	Keep fitted values fixed
Batch normalization	Batch statistics during training	Running estimates during evaluation
Layer normalization	Features within each example	No batch running-average dependency
Group normalization	Channel groups within each example	Independent of batch size

Batch, layer, and group normalization all add a learned scale and shift after normalizing.

Batch, layer, and group norm share statistics differently

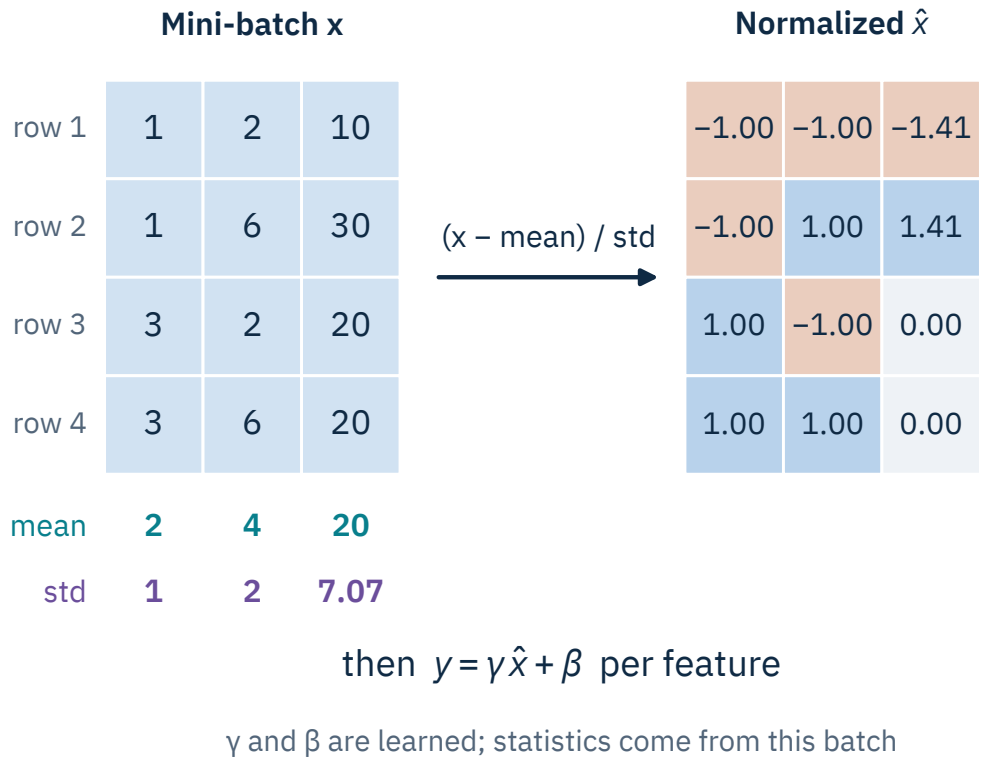


Schematic after Wu and He (2018). For a dense layer H, W = 1: batch norm normalizes each feature over the batch; layer norm normalizes each example over its features.

Batch normalization includes learned scale and shift

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y = \gamma \hat{x} + \beta$$

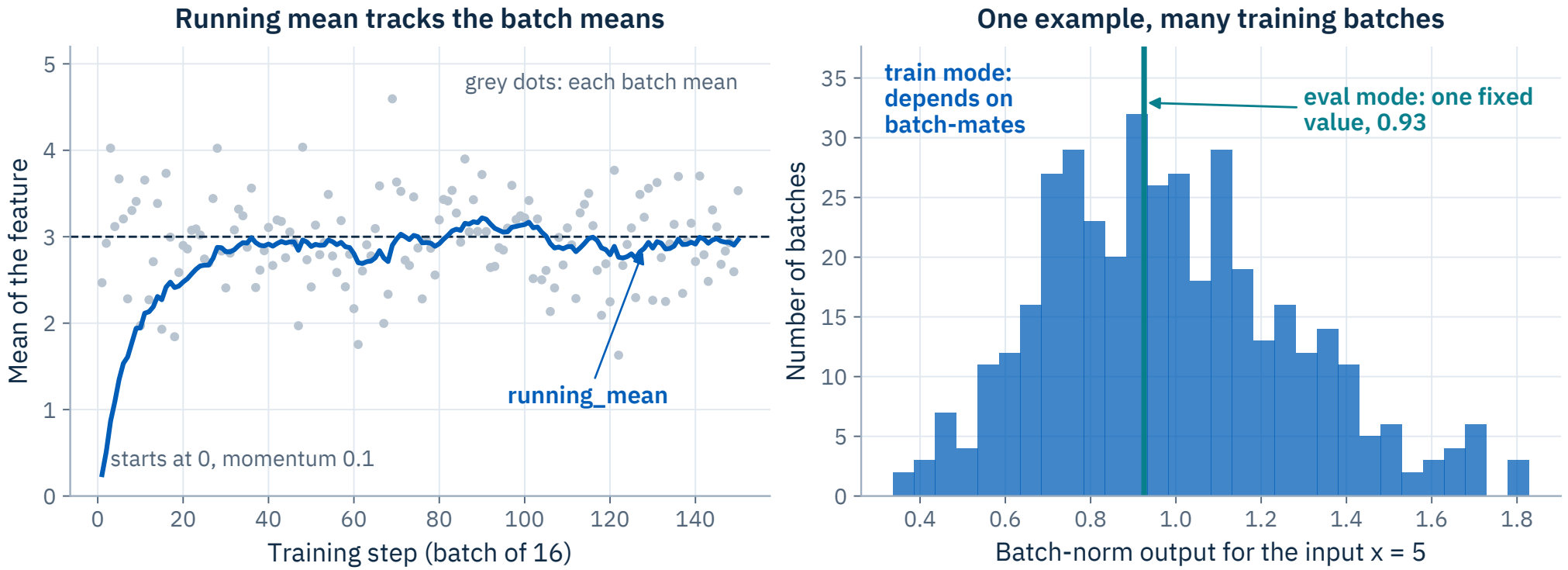
- μ_B and σ_B^2 are computed over the batch for each feature
- γ and β are learned, so the layer can undo the scaling
- Evaluation uses running estimates instead



Exact arithmetic with torch.nn.BatchNorm1d in training mode (affine off); population standard deviation over the batch.

BatchNorm uses batch statistics only in training mode

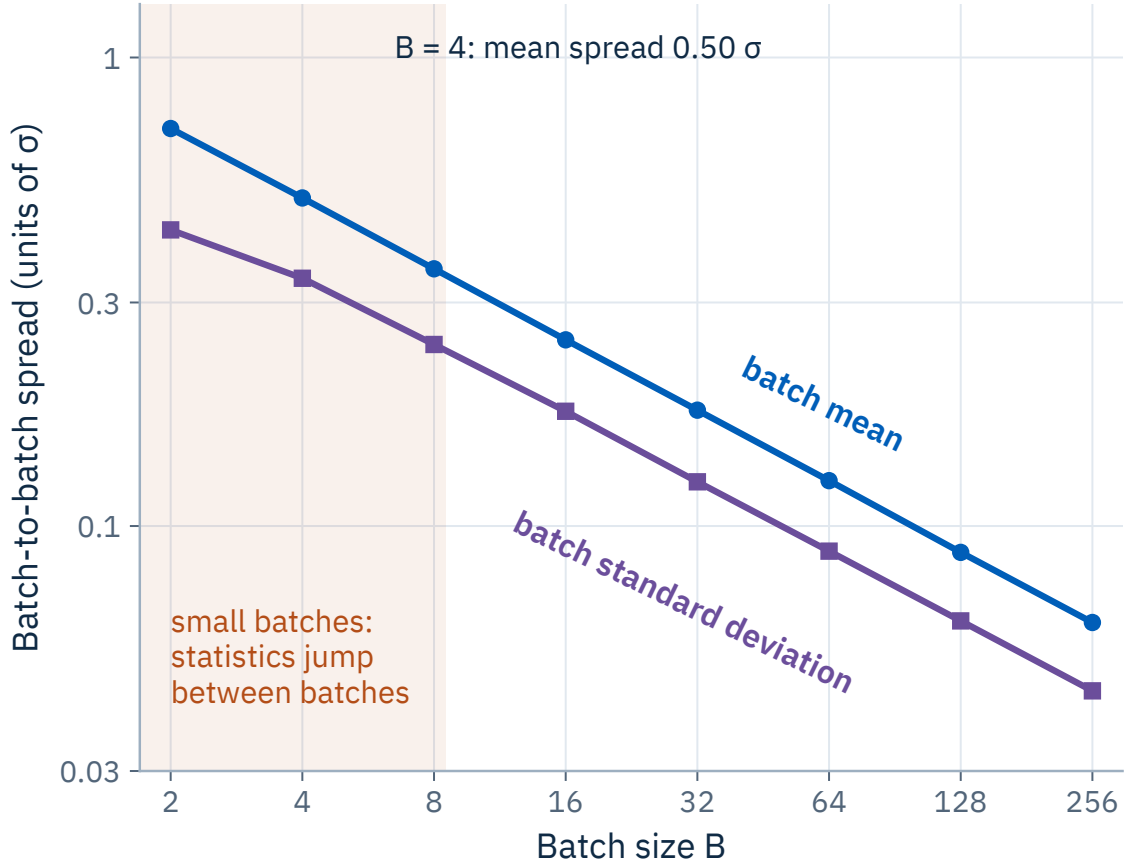
Running averages are updated in training and used after `model.eval()`; forget `eval()` and predictions depend on batch-mates.



Computed with torch.nn.BatchNorm1d (momentum 0.1) on a synthetic feature with mean 3 and standard deviation 2; batches of 16; numpy seed 0.

Batch statistics are noisy when batches are small

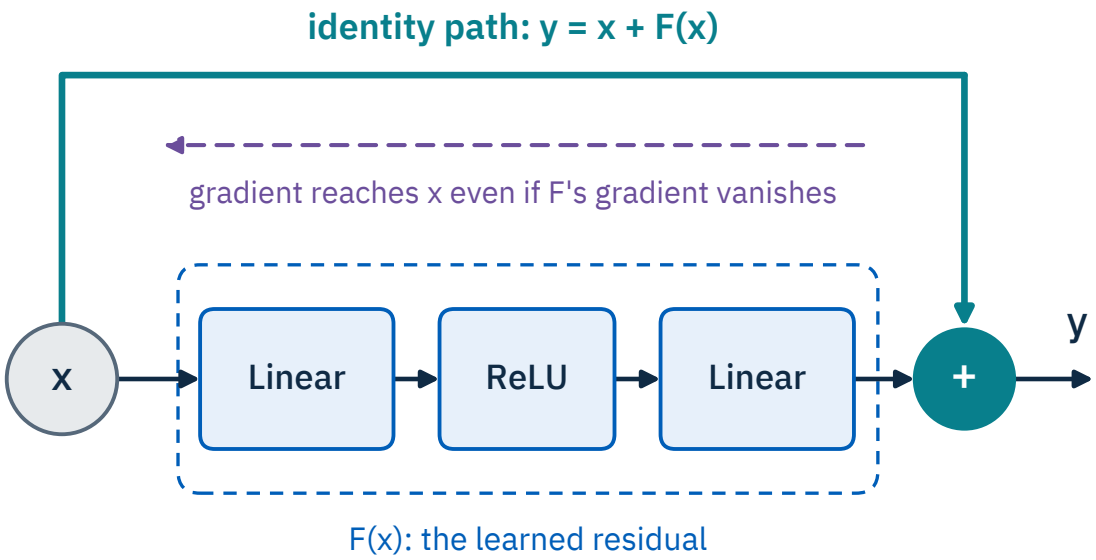
- Batch norm needs large, representative batches (common in Day 4 CNNs)
- With 2 to 8 rows per batch, the mean and variance jump between batches
- Layer and group norm use one example: fine for small batches and Day 5 sequences
- None of them replaces input standardization fitted on training rows



Computed: 20,000 random batches per size from a standard-normal feature, numpy seed 0; spread across batches in units of σ.

Residual connections give gradients a highway

- The block learns a correction $F(x)$ added to its input
- The identity term gives early layers a direct gradient path
- Very deep networks become trainable; Day 4's ResNets build on this
- x and $F(x)$ must have the same shape, or a projection is added



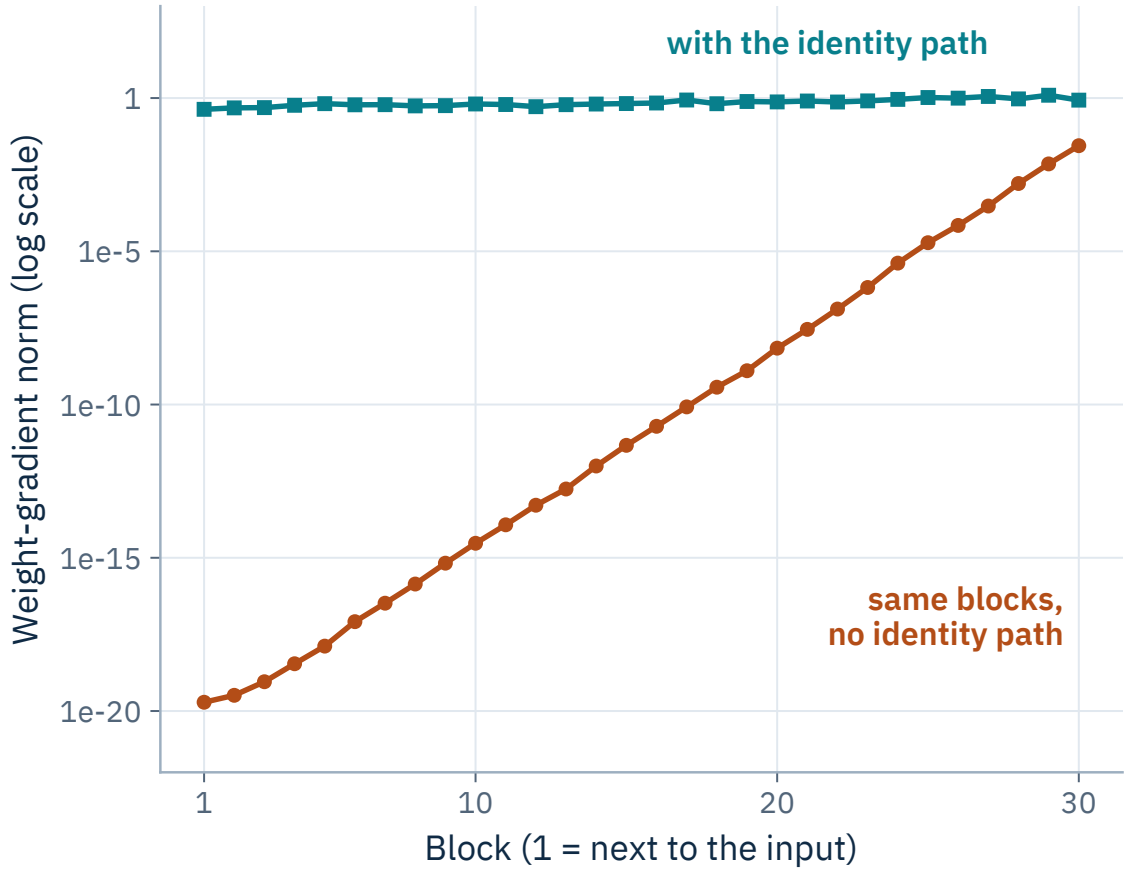
$$\partial y/\partial x = I + \partial F/\partial x$$

Day 4 uses this block in residual CNNs

Schematic residual block: y = x + F(x).

Skip paths keep gradients alive in deep stacks

- Same 30 blocks and identical weights; only the identity path differs
- Without it, the first block's gradient is about 10^{-20}
- With it, every block receives a gradient of similar size



Computed: 30 Linear-ReLU-Linear blocks of width 64, PyTorch default initialization, one backward pass on 256 random rows; torch seeds 0 and 1.

PRACTICE

5 min · Pairs

Which fix would you try first?

For each observation, name the first change you would test and the evidence that would confirm it.

1. Activation histograms collapse to a spike at zero by layer 6.
2. A BatchNorm model gives different predictions for the same well-log sample depending on its batch.
3. A 40-layer plain MLP trains no better than a 4-layer one.

PART 3 OF 5

03

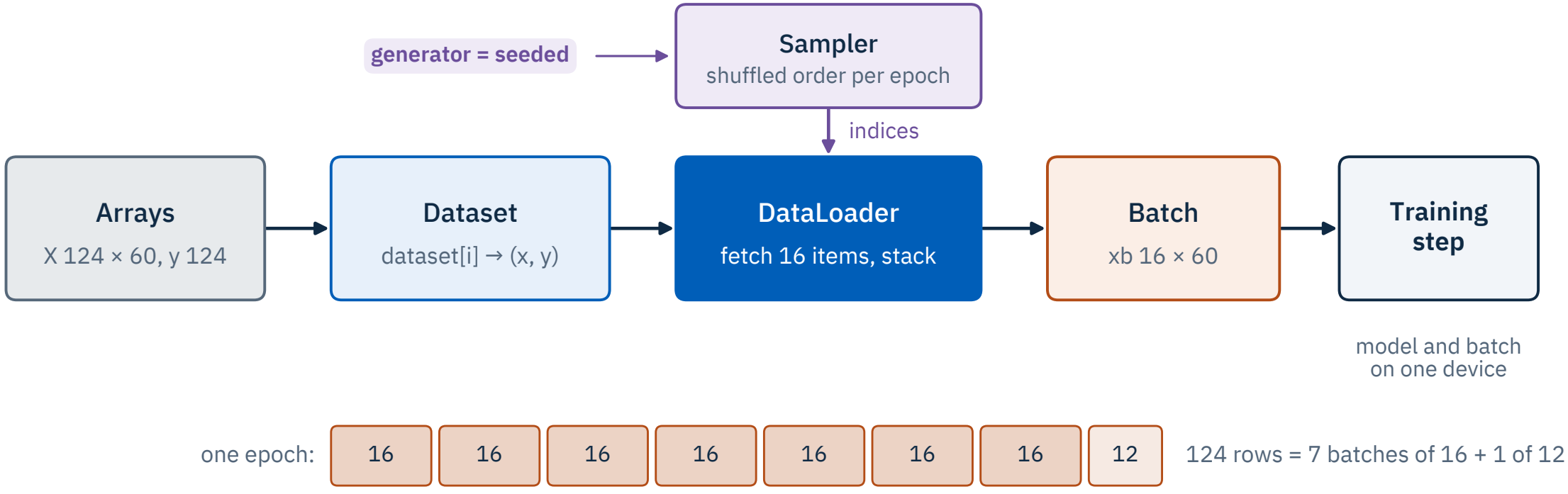
A PyTorch training loop you can trust

Silent errors live in the loop; make each step explicit.

- Datasets, loaders, and devices
- The training step, evaluation mode, and gradients
- Checkpoints, saving, and reloading
- Reproducibility, experiment logs, and the Keras equivalent

Dataset and DataLoader turn arrays into batches

The Dataset answers "give me row i"; the DataLoader decides order, batching, and collation.



Schematic. Sizes follow the Lab 1 Sonar training partition: 124 rows, batch size 16.

Build loaders with an explicit, seeded shuffle

```
from torch.utils.data import (
    TensorDataset, DataLoader)

train_ds = TensorDataset(Xt, yt)
g = torch.Generator().manual_seed(42)
train_loader = DataLoader(
    train_ds, batch_size=16,
    shuffle=True, generator=g)
val_loader = DataLoader(
    TensorDataset(Xv, yv),
    batch_size=64, shuffle=False)
xb, yb = next(iter(train_loader))
print(xb.shape, yb.shape, len(train_loader))
```

OUTPUT

```
torch.Size([16, 60]) torch.Size([16, 1]) 8
```

- Shuffle training batches, never validation
- A seeded generator repeats the batch order
- `len(loader)` is the number of batches per epoch

Model and batches must share one device

```
if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")

model = model.to(device)   # before the optimizer
for xb, yb in train_loader:
    xb = xb.to(device)
    yb = yb.to(device)
```

- Move the model once, before building the optimizer
- Move every batch inside the loop
- Use `.cpu()` before converting results to NumPy
- Small tabular models are often fastest on CPU

A small binary model in PyTorch

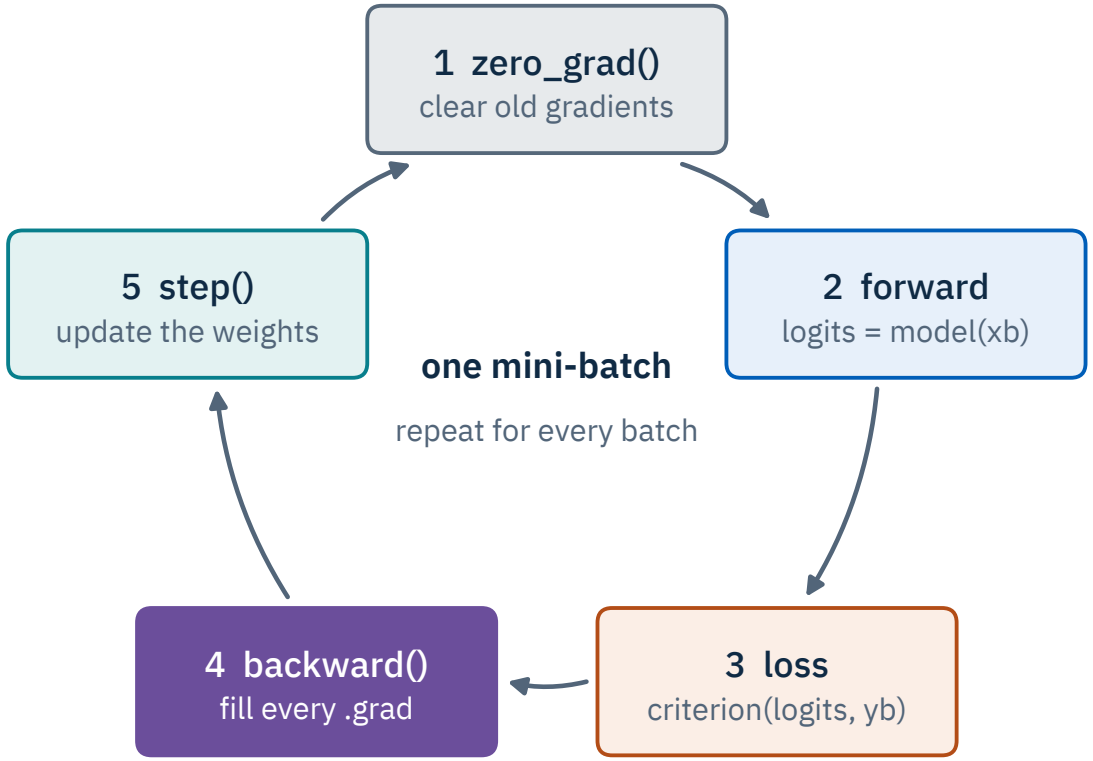
```
import torch
from torch import nn

model = nn.Sequential(
    nn.Linear(n_features, 32), nn.ReLU(),
    nn.Dropout(0.2), nn.Linear(32, 1)
)
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.AdamW(
    model.parameters(), lr=1e-3, weight_decay=1e-3
)
```

The final layer emits one raw logit per row; the loss applies the sigmoid internally.

Every training step repeats five operations

- `zero_grad` clears gradients left from the previous batch
- Forward and loss build the graph on the current batch
- `backward` fills `.grad`; `step` applies the optimizer rule
- Skip or reorder one and training fails quietly



Schematic of one PyTorch optimization step.

The training loop orders zero_grad, backward, and step

```
model.train()
for xb, yb in train_loader:
    xb, yb = xb.to(device), yb.to(device)
    optimizer.zero_grad(set_to_none=True)
    logits = model(xb)                          # (B, 1)
    loss = criterion(logits, yb)                # yb: (B, 1) floats
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
```

Clipping is optional; if used, it belongs after `backward()` and before `step()`.

Evaluation mode and gradient tracking are separate

```
model.eval()
with torch.inference_mode():
    logits = model(X_val.to(device))                    # (n, 1)
    val_loss = criterion(logits, y_val.to(device))      # y_val: (n, 1)
    probabilities = torch.sigmoid(logits).squeeze(-1).cpu().numpy()
```

- `eval()` changes dropout and normalization behaviour
- `inference_mode()` switches off autograd bookkeeping for inference

Mode switches and autograd switches do different jobs

Call	Dropout	BatchNorm statistics	Gradient tracking
`train()`	active	batch; running averages updated	no effect
`eval()`	off	running averages used	no effect
`no_grad()`	no effect	no effect	off
`inference_mode()`	no effect	no effect	off; stricter, faster

Evaluation needs both: `eval()` for layer behaviour, and a no-grad context for speed and memory.

A reference to state_dict is not a snapshot

```python
import copy

best_ref = model.state_dict()                    # live references
best_copy = copy.deepcopy(model.state_dict())    # snapshot
w0 = best_copy["0.weight"][0, 0].item()

train_one_epoch(model, train_loader, optimizer)   # weights change

print(f"{w0:.4f}")
print(f"{best_ref['0.weight'][0, 0].item():.4f}")
print(f"{best_copy['0.weight'][0, 0].item():.4f}")
```

OUTPUT

```
0.0987
0.0919
0.0987
```

Early stopping needs a copied checkpoint

```
import copy

best_loss, stale, patience = float("inf"), 0, 10
for epoch in range(1, max_epochs + 1):
    train_one_epoch(model, train_loader, optimizer)
    val_loss = evaluate(model, val_loader)       # eval mode inside
    if val_loss < best_loss:
        best_loss, best_epoch, stale = val_loss, epoch, 0
        best_state = copy.deepcopy(model.state_dict())
    else:
        stale += 1
        if stale >= patience:
            break

model.load_state_dict(best_state)                # restore, then
model.eval()                                     # evaluate or save
```

Save the state dict with everything needed to use it

```
torch.save(model.state_dict(), run_dir / "weights.pt")
joblib.dump(scaler, run_dir / "scaler.joblib")
(run_dir / "run.json").write_text(json.dumps(metadata, indent=2))

restored = build_model(config["width"], config["dropout"])
state = torch.load(run_dir / "weights.pt",
                   map_location="cpu", weights_only=True)
restored.load_state_dict(state)
restored.eval()
check = torch.sigmoid(restored(X_val_scaled)).detach().numpy().ravel()
np.testing.assert_allclose(check, val_prob, rtol=1e-6, atol=1e-6)
```

The reload check proves the saved files reproduce the predictions you validated.

Save the whole prediction path, not only the weights

A weights file without its scaler, feature order, threshold, and split is not a model anyone can use.

Operations will run your model months later on new data; everything that turns raw sensor rows into a decision must travel with it.

After training: restore, verify, save, test once

Five steps separate the last epoch from a result you can report.

1. **Restore** — load the best validation checkpoint
2. **Verify** — eval mode and a reload prediction check
3. **Save** — weights, scaler, config, threshold, split
4. **Test once** — locked final assessment, no tuning
5. **Report** — metrics with budget, seeds, and limits

◆ If a test result changes your settings, the test set has become a validation set.

Mixed precision is a GPU speed option, not a default

```
use_amp = device.type == "cuda"
for xb, yb in train_loader:
    xb, yb = xb.to(device), yb.to(device)
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(
            device_type=device.type,
            dtype=torch.bfloat16,
            enabled=use_amp):
        loss = criterion(model(xb), yb)
    loss.backward()
    optimizer.step()
```

- Autocast runs matrix multiplies in 16-bit where safe
- bfloat16 needs no loss scaling; float16 needs a GradScaler
- Compare losses and metrics with a float32 run
- Little benefit for small tabular models

Log enough to rerun and audit every experiment

Configuration

- Architecture, optimizer, learning rate, schedule
- Batch size, epochs, patience, seeds

Data

- Dataset version and split indices
- Feature list and fitted preprocessing

Curves

- Training and validation loss per epoch
- Learning rate and gradient norm per step

Environment

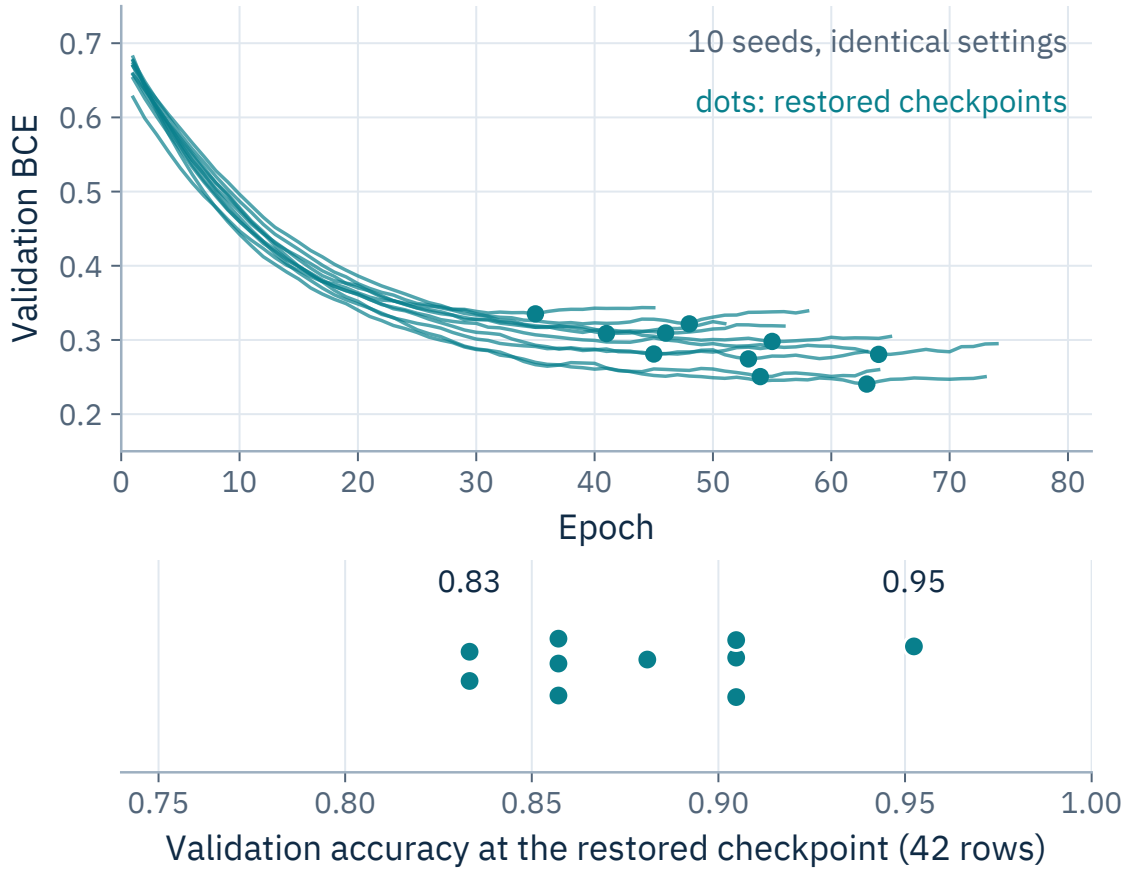
- Library versions and device
- Code version and run time

Outcome

- Selected epoch and checkpoint path
- Decision and reason, even for failed runs

Reproducibility includes the experiment definition

- Record split indices, seeds, dataset version, and feature schema
- Save preprocessing, configuration, weights, and threshold
- Log library versions and hardware
- Repeat promising settings over several seeds



◆ Ten seeds, one configuration: validation accuracy from 0.83 to 0.95 on 42 rows.

Computed from the course file sonar.all-data.csv with the Lab 1 split and training loop; torch seeds 0–9.

The same contract in Keras

```
import keras

model = keras.Sequential([
    keras.layers.Input(shape=(n_features,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dropout(0.2), keras.layers.Dense(1)
])
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.BinaryCrossentropy(from_logits=True)
)
```

Same architecture, same raw-logit output, same loss; `fit()` hides the loop you just wrote.

Explicit validation is easier to audit

```
stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10,
    restore_best_weights=True
)
model.fit(
    X_train_scaled, y_train,
    validation_data=(X_val_scaled, y_val),
    epochs=100, batch_size=32, callbacks=[stop]
)
```

Prefer `validation_data` over `validation_split`: you choose the rows, and the scaler never sees them.

LABORATORY

60 min

Laboratory 1: a reproducible PyTorch pipeline

- ☐ Assert that the scaler was fitted on the 124 training rows only
- ☐ Predict the parameter count for width 32, then check it
- ☐ Mark the five training-step operations, the train/eval switch, and the copy
- ☐ Change one setting, keep split and budget fixed, compare with the baseline
- ☐ Save, reload, pass the prediction check; keep `RUN_FINAL = False`

NOTEBOOK

`exercises/01_pytorch_pipeline/PyTorch_Pipeline_Student.ipynb`

EVIDENCE TO SUBMIT

- Learning curves with the selected epoch
- Config, checkpoint, and reload evidence
- Baseline comparison and one error case

◆ Keras track: 02_tensorflow_pipeline, only where TensorFlow is installed.

DISCUSSION

Lab 1 debrief: what the evidence supports

Reference run: seed 42 on CPU; identical in PyTorch 2.3 and 2.14, may vary elsewhere.

1,985

parameters for width 32

epoch 44

restored checkpoint (stopped at 54)

0.86 vs 0.69

validation accuracy, network vs logistic baseline

- 42 validation rows: one row moves accuracy by 2.4 points, and the checkpoint was chosen on them.

PART 4 OF 5

04

Optimizers, schedules, and tuning

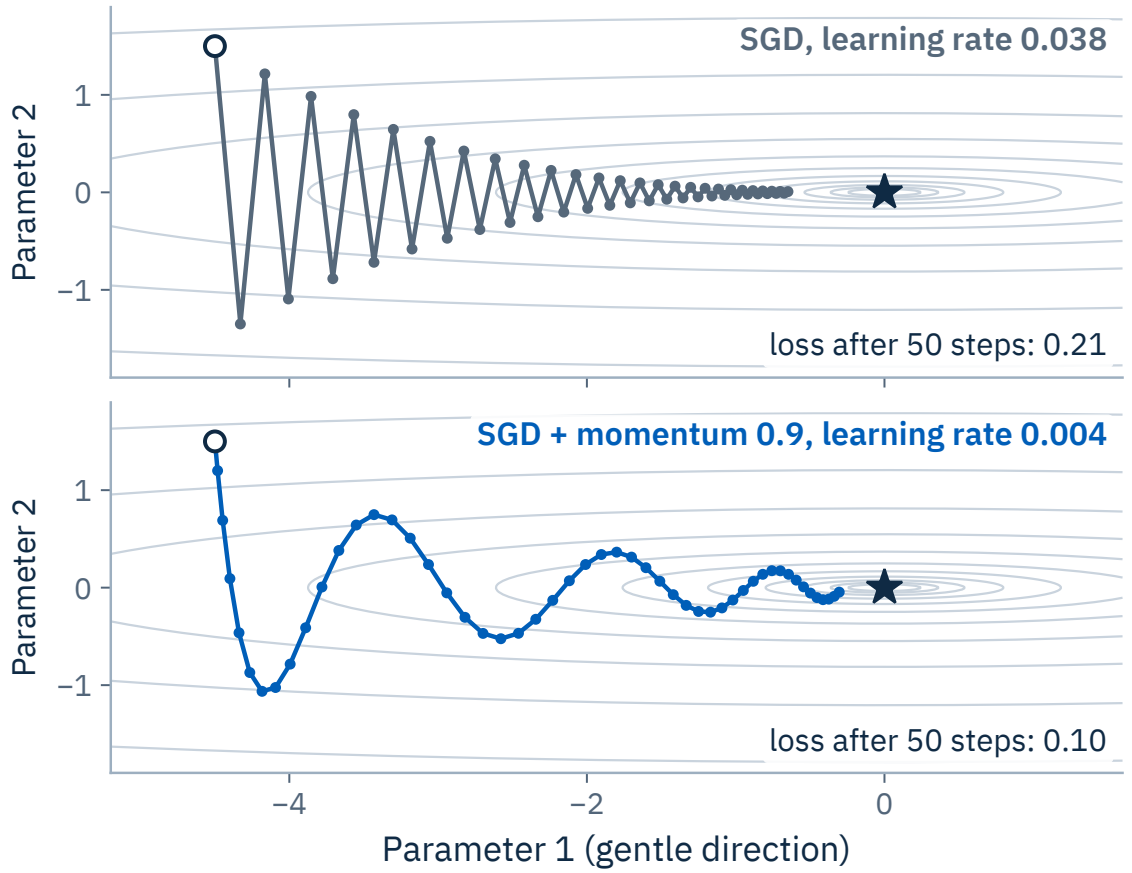
Learning rate often matters more than a large architecture search.

- Momentum and adaptive optimizers
- AdamW and decoupled weight decay
- Learning-rate range tests and schedules
- Batch size, clipping, and a tuning budget

SGD with momentum accumulates direction

$$v_t = \beta v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t$$

- *v* is a running sum of past gradients, weighted by β
- Zig-zags across the valley cancel; steady progress along it adds up
- The learning rate still sets the overall step scale



Computed with torch.optim on L = (w1² + 50 w2²)/2, 50 steps from (−4.5, 1.5); learning rates chosen for illustration.

Adaptive optimizers track gradient scale

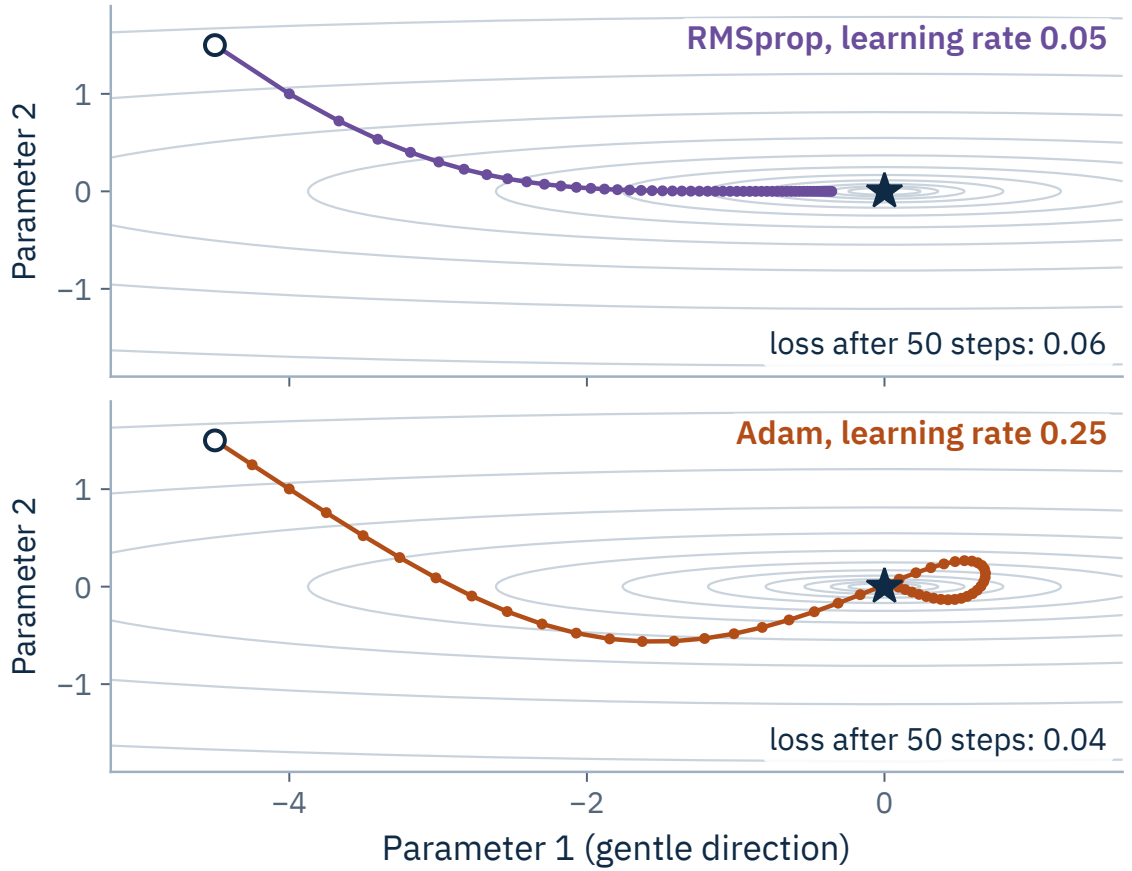
Optimizer	Running information	Practical consideration
RMSprop	Squared gradients	Adapts step sizes per parameter
Adam	Gradients and squared gradients	Bias-corrected moment estimates
AdamW	Adam-style moments plus decoupled decay	Separates decay from adaptive gradient scaling

Each still has a learning rate that must be tuned; defaults are starting points.

Adam's update uses two moment estimates

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- *m* tracks direction and *v* tracks squared size, per parameter
- Hats mark the correction for averages that start at zero
- Early steps are about η per parameter, whatever the gradient size
- ε keeps the denominator away from zero



Computed with torch.optim (default betas and alpha) on the same bowl; learning rates chosen for illustration.

A fair comparison tunes each optimizer's learning rate

An optimizer run at someone else's learning rate is not a fair competitor.

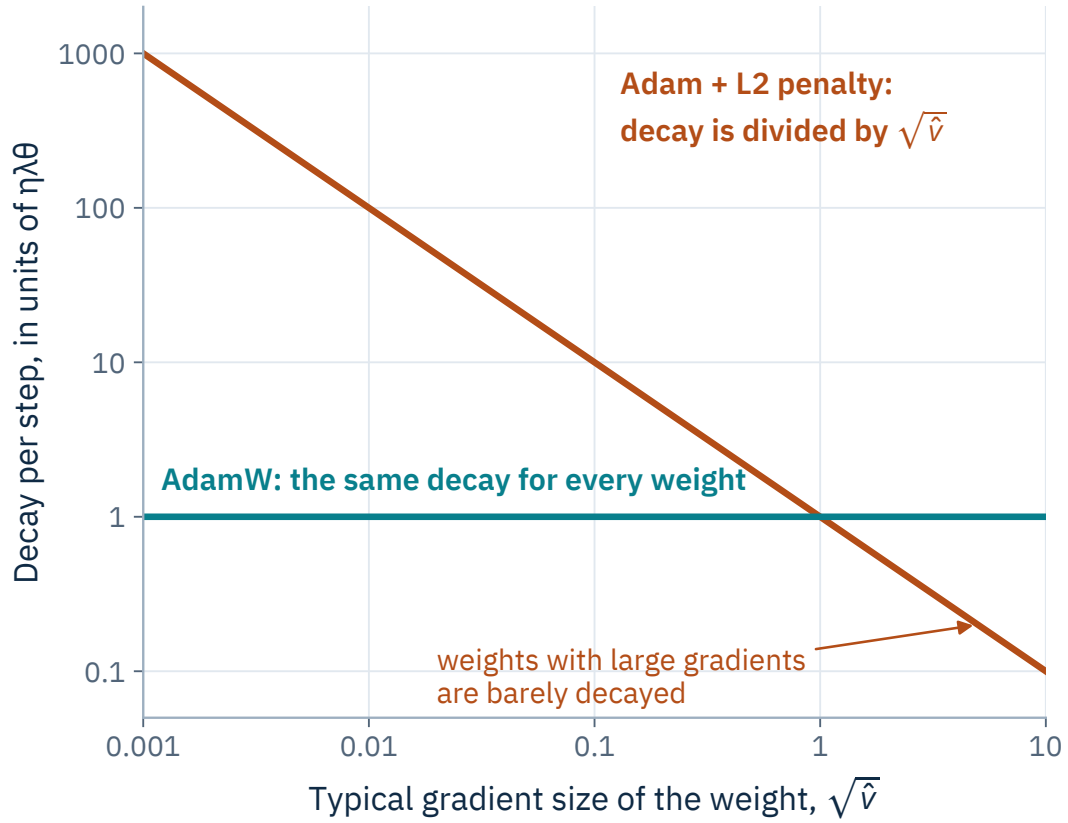
Give every candidate the same budget, the same split, several seeds, and its own small learning-rate search.

AdamW decouples weight decay from the adaptive step

$$\text{Adam} + \text{L2}: \; g_t \leftarrow g_t + \lambda\theta_t \text{ before scaling}$$

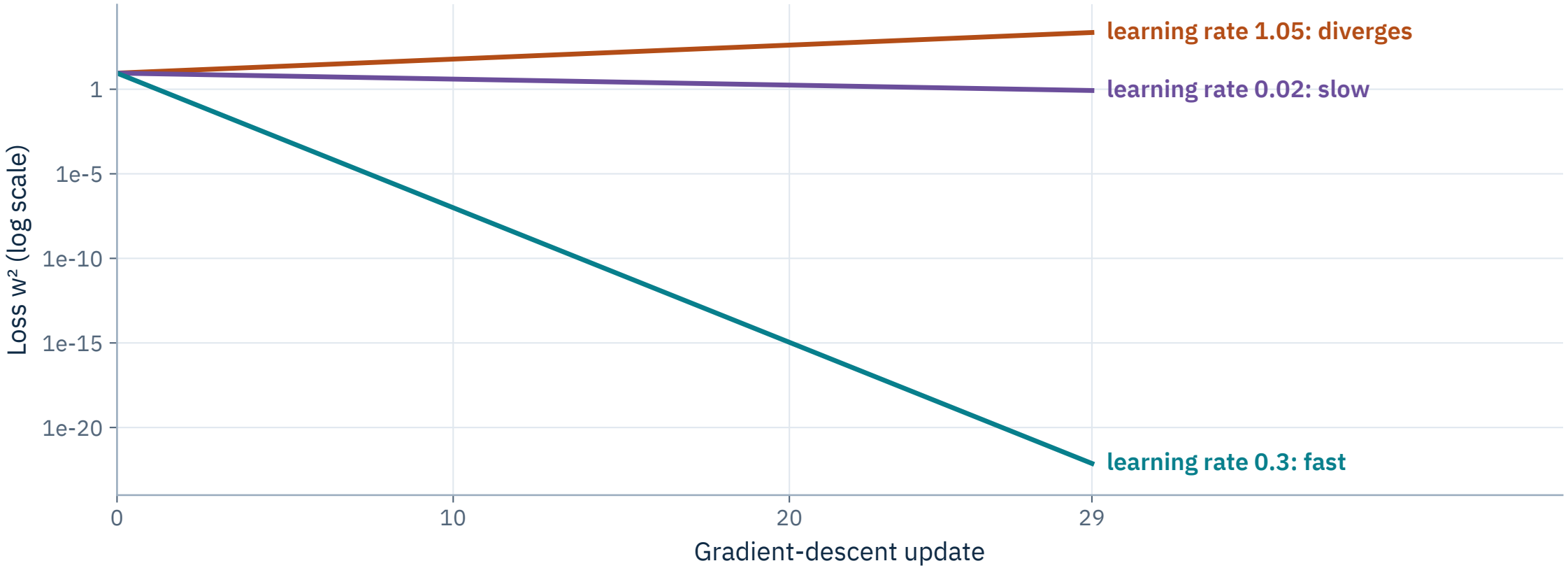
$$\text{AdamW}: \; \theta_{t+1} = \theta_t - \eta\left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\theta_t\right)$$

- With L2 inside Adam, the adaptive denominator divides the penalty too
- Weights with large gradients are then barely regularized
- `Adam(weight_decay=...)` is the L2 form; `AdamW` is decoupled



Analytical approximation, valid while the decay term is small relative to the gradient.

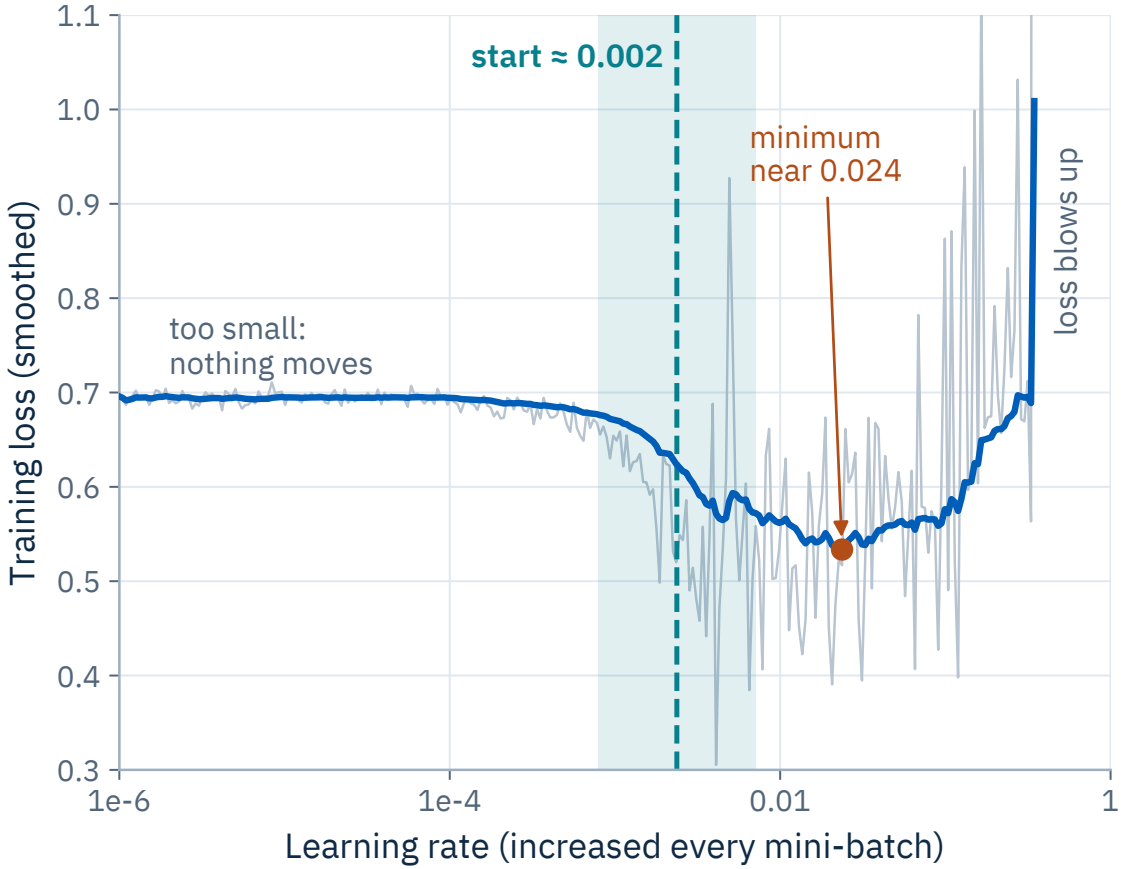
Step size changes convergence



Computed: exact gradient descent on the loss w^2 from $w = 3$ with three learning rates. Large steps can diverge even in a simple problem.

A learning-rate range test finds the usable window

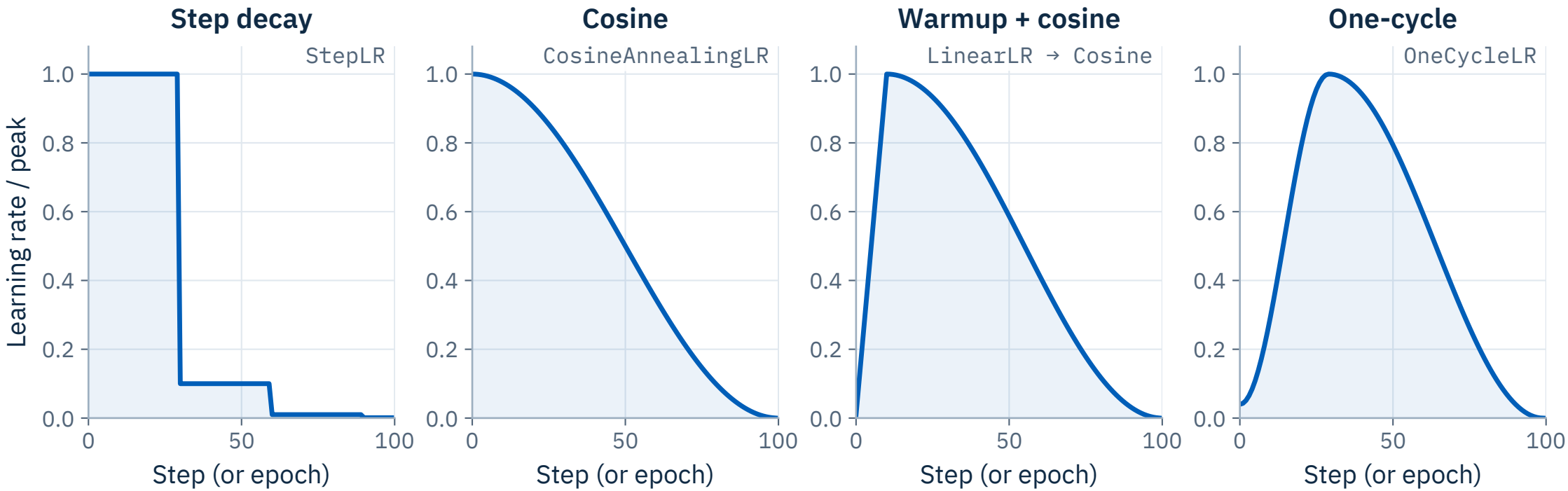
- Raise the rate every mini-batch, starting from 1e-6
- Stop when the smoothed loss climbs well above its start
- A common heuristic: start about 10× below the loss minimum
- Rerun after changing batch size, model, or optimizer



Computed: synthetic ESP table (4,000 rows, seed 3), MLP 12-64-64-1, AdamW, batch 32, torch seed 0; grey = raw batch loss, blue = moving average.

Schedules are part of the experiment

Step, cosine, warmup, and one-cycle schedules shape the same peak rate over a fixed budget.



Computed by stepping the named torch.optim.lr_scheduler classes for 100 steps; peak rate scaled to 1.

Schedulers step on a declared clock

```python
from torch.optim import lr_scheduler as S

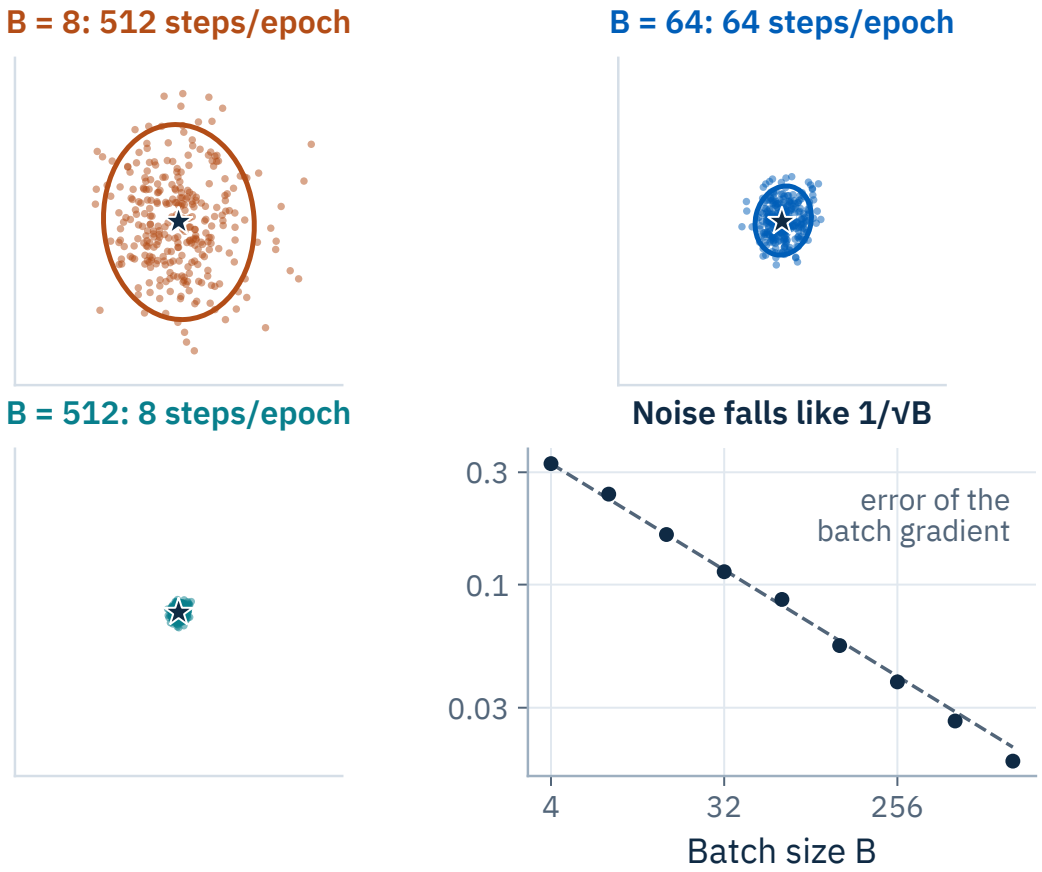
steps = epochs * len(train_loader)
warm = steps // 10
warmup = S.LinearLR(optimizer, start_factor=0.01, total_iters=warm)
cosine = S.CosineAnnealingLR(optimizer, T_max=steps - warm)
sched = S.SequentialLR(optimizer, [warmup, cosine], milestones=[warm])

for epoch in range(epochs):
    for xb, yb in train_loader:
        ...                              # zero_grad, forward, backward
        optimizer.step()
        sched.step()                     # once per batch: the clock
```

`ReduceLROnPlateau` is the exception: call `sched.step(val_loss)` once per epoch.

Batch size changes both compute and learning

- Larger batches may improve hardware throughput
- Smaller batches add gradient noise and more updates per epoch
- Memory depends on activations as well as parameters
- Compare update counts or compute budgets, not epochs alone



Computed: logistic-regression gradients at one fixed point, 4,096 synthetic rows (numpy seed 0), 300 random mini-batches per size.

Bigger batches mean fewer updates per epoch

If all 3,243 rows of the Day 3 ESP training file were used for training, one epoch would give:

203

updates at batch size 16

51

updates at batch size 64

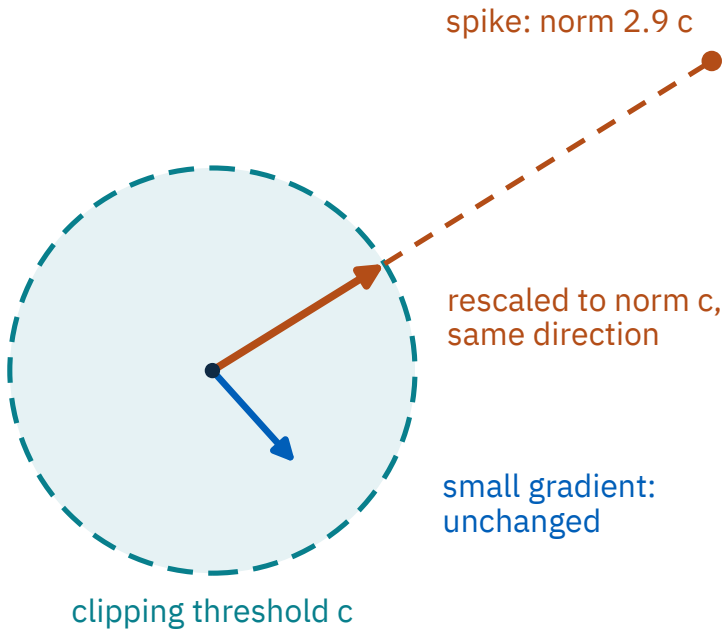
13

updates at batch size 256

- Doubling the batch halves the updates per epoch; compare runs at matched updates or compute.

Gradient clipping caps rare, oversized steps

- Rescales the whole gradient when its global norm exceeds c
- Direction is kept; only the step length is capped
- Call after `backward()`, before `optimizer.step()`
- Log how often it triggers; constant clipping hides a bad rate



$$g \leftarrow g \cdot \min(1,\ c / \|g\|)$$

one global norm over all parameters

Schematic of gradient-norm clipping in two dimensions.

PRACTICE

6 min · Pairs

Plan a fair optimizer comparison

You will compare SGD with momentum against AdamW on the ESP table (3,243 training rows).

1. With batch size 32 for 50 epochs, how many optimizer updates is that?
2. A colleague switches to batch size 64 but keeps 50 epochs. What changed besides the batch?
3. What must be tuned separately for each optimizer before comparing them?

Tune the settings that matter most first

1 · Learning rate and schedule

- Search on a log scale first
- Often the most influential setting

2 · Batch size with the rate

- Change them together
- Match the update budget

3 · Regularization strength

- Weight decay, dropout, patience
- Tune once the rate is sane

4 · Width and depth

- Only if training loss stays high
- Grow in steps of about 2×

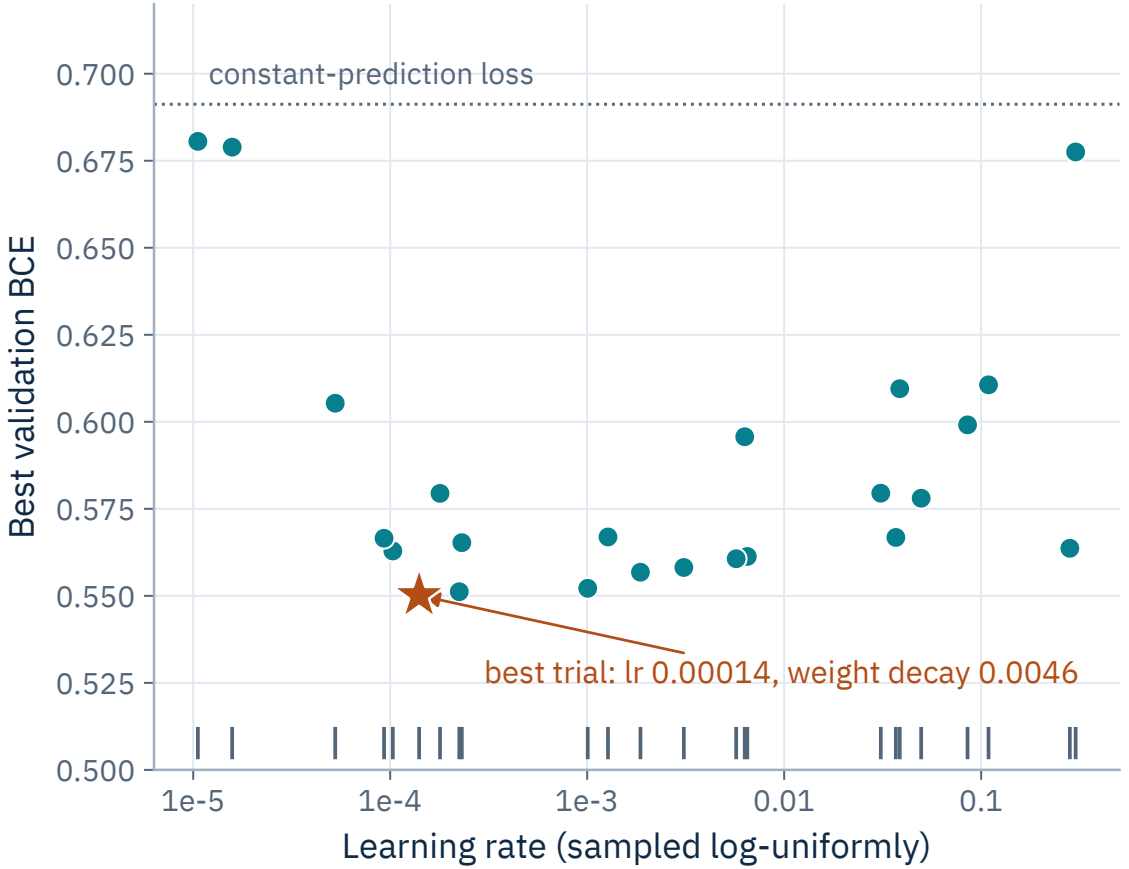
5 · Everything else

- Optimizer betas, epsilon, init details
- Usually fine at defaults

◆ Search strategies (grid, random, successive halving, Bayesian) are covered in ML Module 5.

Sample learning rates on a log scale

- Draw log10 of the rate uniformly: each decade gets equal attention
- A wide first pass finds the usable window; then narrow it
- Losses within about 0.01 may be seed noise: repeat the best few
- Keep split, budget, and seed policy fixed across trials



Computed: 24 random trials of learning rate and weight decay (numpy seed 7) on the synthetic ESP table; best validation loss with patience 15.

PRACTICE

6 min · Groups

A small search can answer a useful question

You can afford twelve training runs on the Lab 1 Sonar pipeline.

1. Define the comparison you want to learn from those runs.
2. Choose at most two settings to vary.
3. Specify what remains fixed and how you will select the final checkpoint.

DISCUSSION

A twelve-run plan that answers one question

Runs	What varies	Seeds	Decision rule
1–4	learning rate 3e-4, 1e-3, 3e-3, 1e-2	0	keep the best two
5–12	the best two rates	1–4 each	compare mean and spread
after	final assessment of the chosen setting	—	once, no changes

Fixed throughout: split, width 32, dropout 0.2, weight decay 1e-3, batch 16, 80 epochs, patience 10.

PART 5 OF 5

05

Regularization and autoencoders

Constrain what a network can memorize; then learn without labels.

- Weight decay, dropout, and early stopping
- Augmentation for tabular and sensor data
- Label smoothing and the double-descent curve
- Autoencoders and reconstruction scores

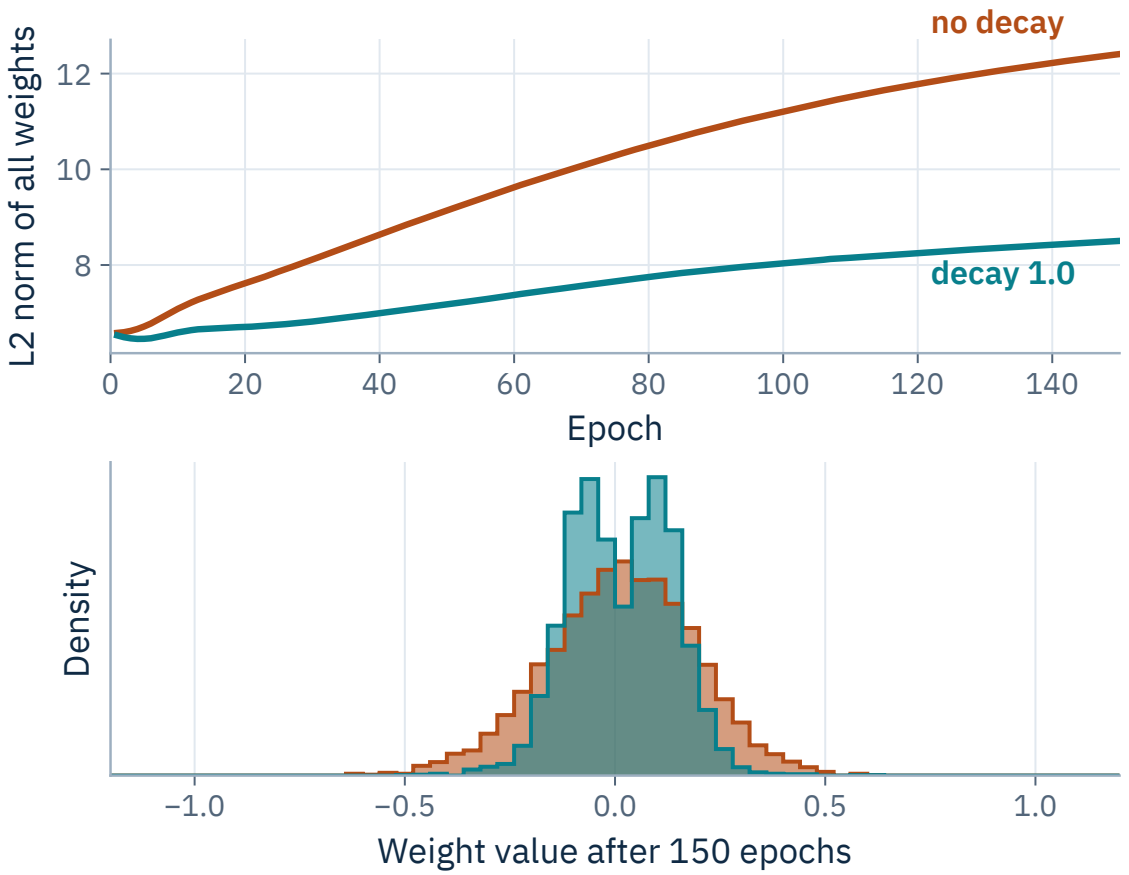
Regularization controls different mechanisms

Technique	Mechanism	What to tune
Weight penalty / decay	Discourages large parameter values	Penalty strength
Dropout	Perturbs hidden activations	Drop probability
Early stopping	Chooses a checkpoint before validation worsens	Patience and monitored metric
Data augmentation	Adds valid training variations	Label-preserving transforms
Label smoothing	Softens one-hot targets	Smoothing amount ε

Weight decay pulls every weight toward zero

$$\tilde{L} = L + \tfrac{\lambda}{2}\|w\|^2 \quad \Rightarrow \quad \nabla\tilde{L} = \nabla L + \lambda w$$

- The penalty adds λ *w* to each gradient; AdamW shrinks *w* directly
- Smaller weights make smoother functions of the inputs
- Here decay 1.0 limited the weight norm to 8.5 instead of 12.4

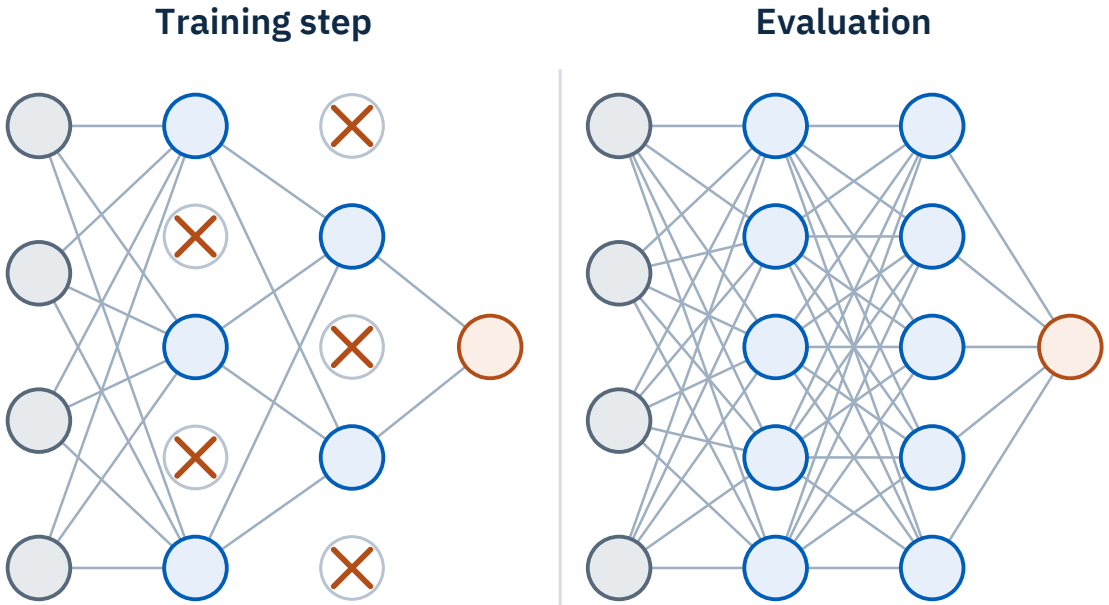


Computed: synthetic ESP table (seeds 1 and 2), MLP 12-64-64-1, AdamW lr 1e-3 with weight_decay 0 or 1.0, 150 epochs, torch seed 0.

Dropout changes training-time activations

$$\tilde{a} = \frac{m \odot a}{1-p}, \qquad m_j \sim \text{Bernoulli}(1-p)$$

- During training, each activation is kept with probability 1–*p*
- Inverted dropout rescales kept activations to preserve their expectation
- During evaluation, dropout is disabled

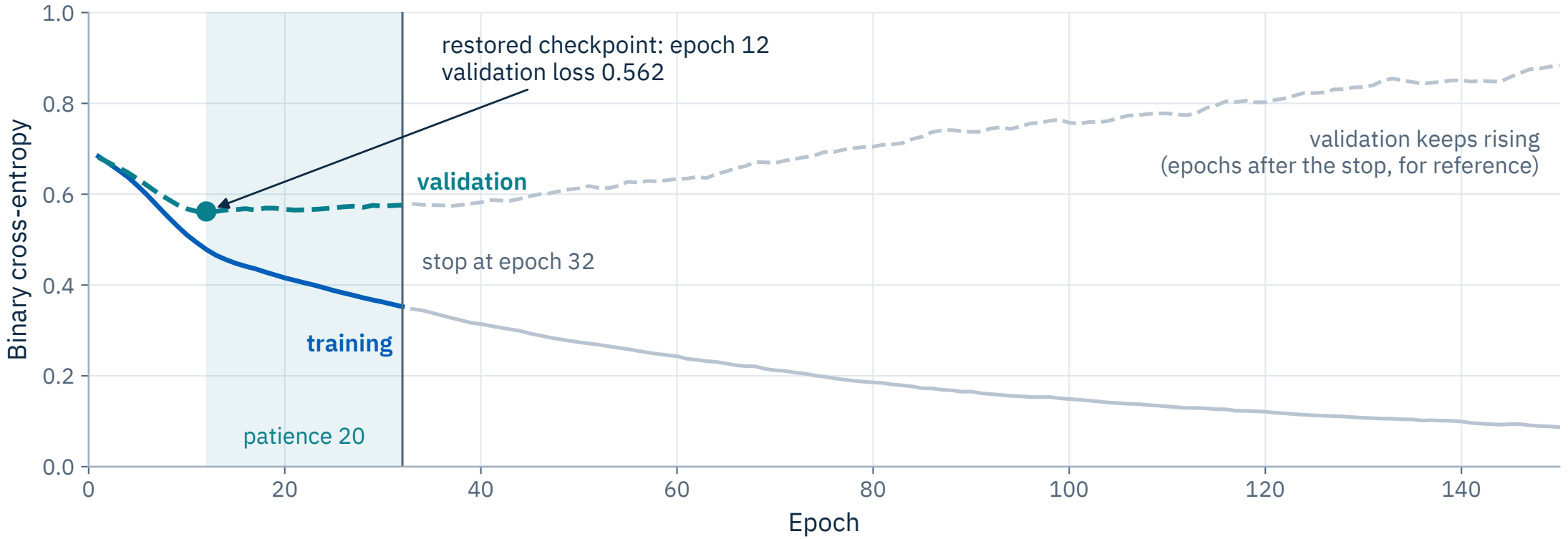


a new random mask is drawn for every mini-batch

Schematic of inverted dropout with drop probability p = 0.5.

Early stopping keeps the best validation checkpoint

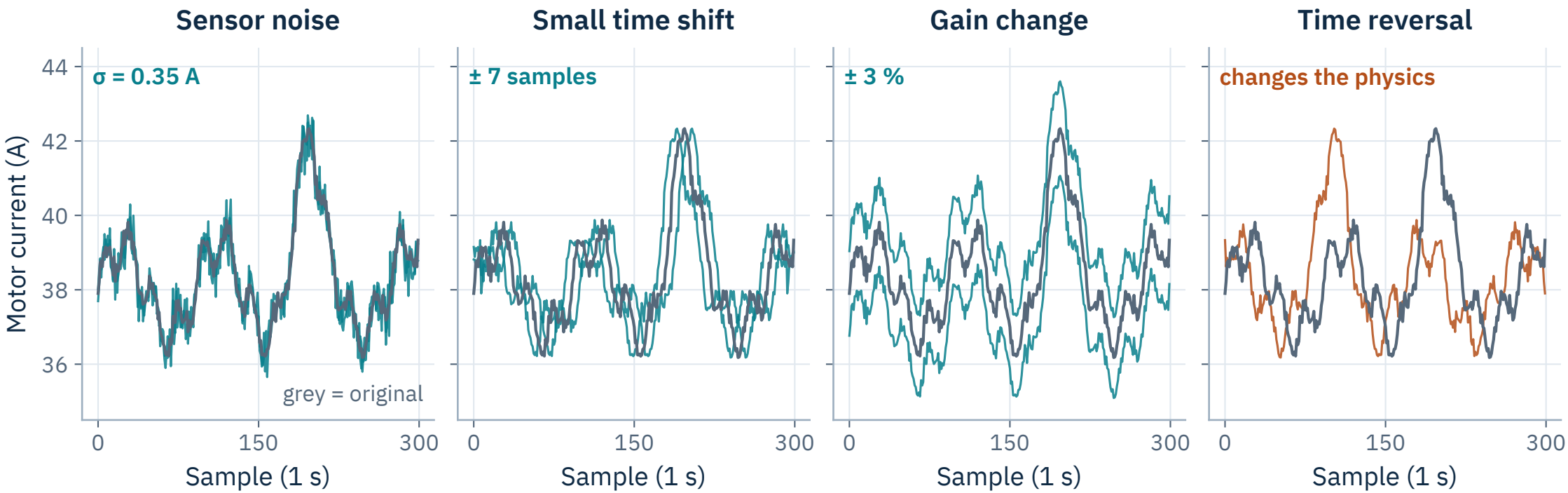
The patience rule decides when to stop; the copied checkpoint decides what you keep.



Computed: synthetic ESP table (seeds 1 and 2), MLP 12-64-64-1, AdamW (weight decay 0.1, dropout 0.2), torch seed 0; patience 20 applied to the recorded curve; grey = epochs after the stop.

Sensor augmentation must preserve the label

Add only variation the real sensors could produce under the same operating condition.



Computed: synthetic ESP motor-current trace (numpy seeds 4 and 1); noise level, shift, and gain are illustrative, not sensor specifications.

Augment training rows only, within physical limits

✓ Reasonable

- Noise at the sensor's stated precision
- Small shifts that keep the event in the window
- Gain changes within calibration tolerance
- Dropping a channel, if real outages occur

vs

✗ Risky

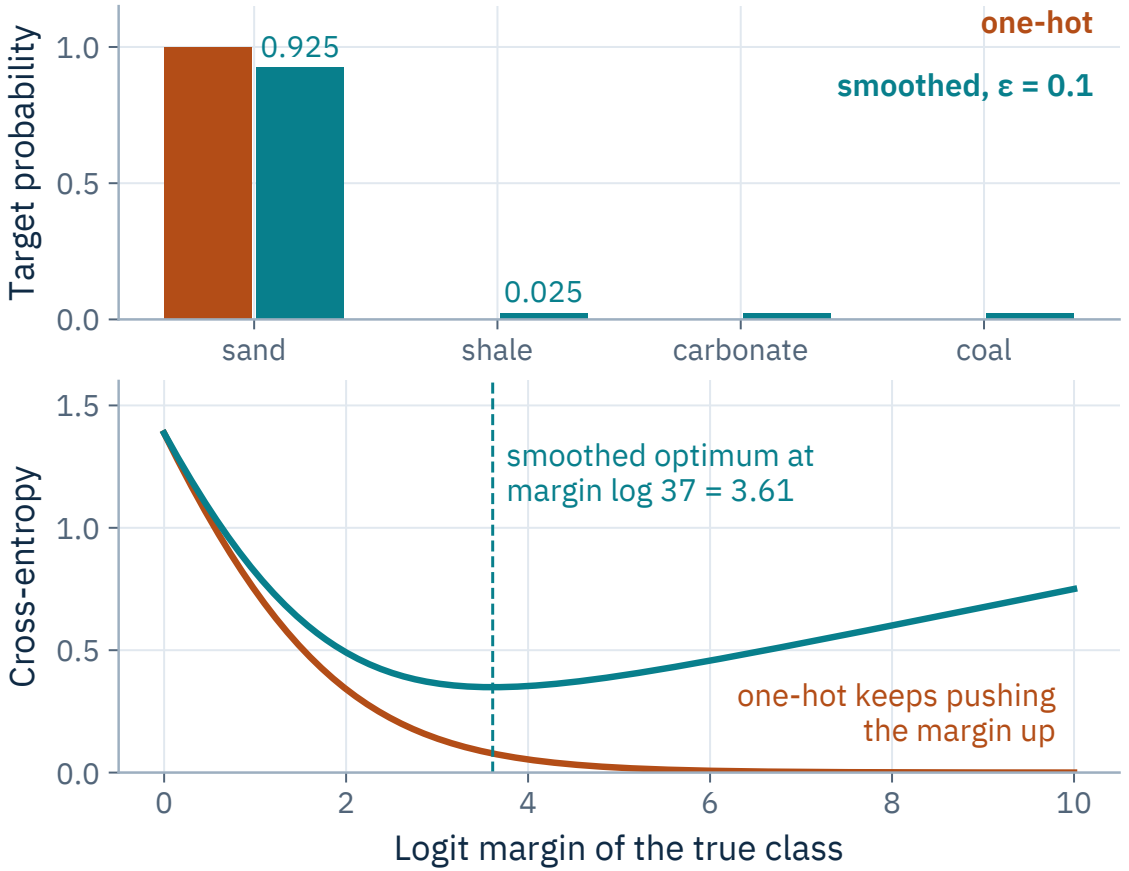
- Augmenting before the split: copies leak
- Values outside physical limits
- Time reversal; mixing wells or regimes
- Augmenting validation or test rows

◆ Augmented copies of a row must stay in the same fold as the original row.

Label smoothing softens overconfident targets

$$y_k^{\mathrm{LS}} = (1-\varepsilon)\, y_k + \frac{\varepsilon}{K}$$

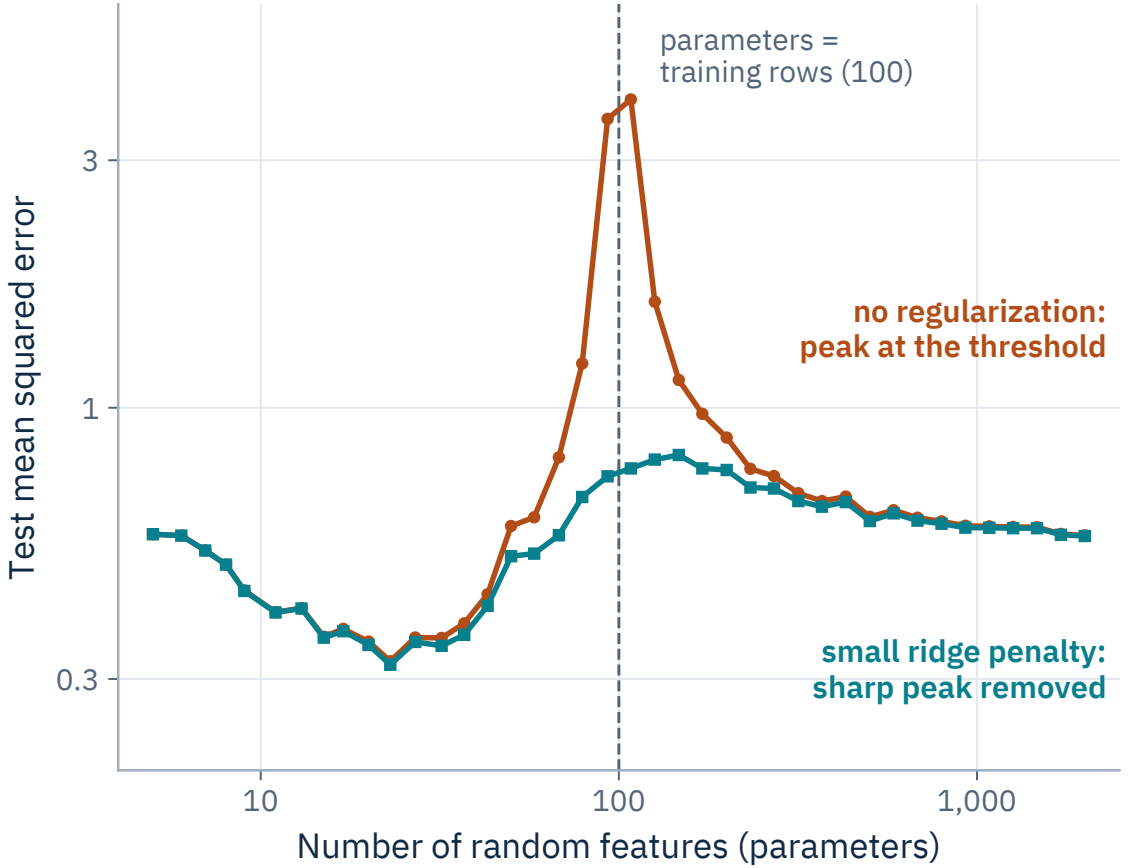
- For K = 4 and ε = 0.1 the targets become 0.925 and 0.025
- The loss no longer rewards ever-larger logits
- PyTorch: `label_smoothing=0.1` in the loss
- Check calibration on validation data; probabilities shift



Exact targets and cross-entropy for four classes with smoothing 0.1; other logits held equal.

Test error can peak where parameters equal rows

- Classical U-curve: error falls, then rises with capacity
- At the threshold, the fit is forced through every noisy row
- Beyond it, error can fall again; a small penalty removes the spike
- Shown for random-feature regression; not a guarantee for your model



Computed: random ReLU features fitted by minimum-norm least squares (orange) or ridge 0.1 (teal) to 100 noisy synthetic rows; numpy seed 0.

PRACTICE

5 min · Pairs

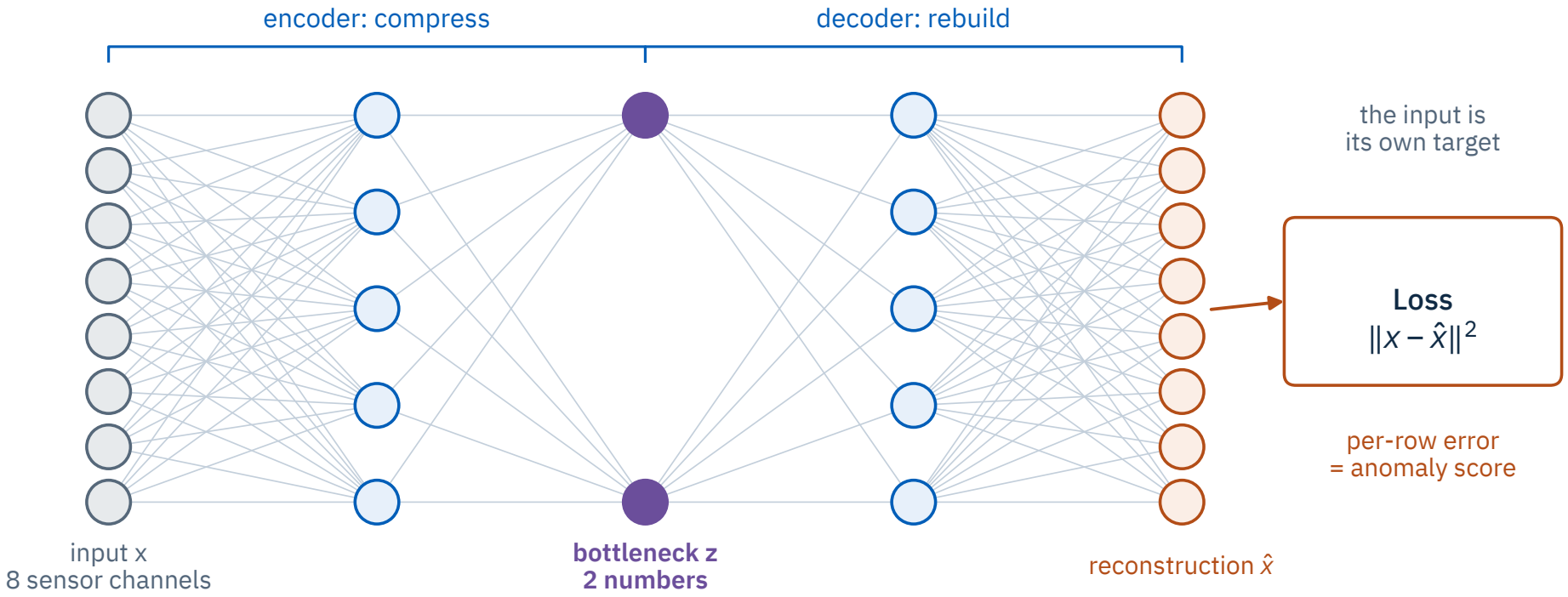
Match the regularizer to the symptom

For each case, choose a first remedy and the evidence you would check afterwards.

1. Training loss 0.05, validation loss rising after epoch 15, only 300 rows.
2. Validation accuracy is fine, but wrong predictions come with probabilities of 0.999.
3. A model trained on one field fails on sensors with a 2% gain offset.

An autoencoder squeezes data through a bottleneck

Trained to reproduce its own input, the network must keep only the dominant structure of the data.



Schematic of a dense autoencoder (8-5-2-5-8); the loss compares the reconstruction with the input.

An autoencoder is an encoder, a decoder, and an MSE loss

```
class AE(nn.Module):
    def __init__(self, n_in=8, k=2):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(n_in, 16), nn.Tanh(), nn.Linear(16, k))
        self.decoder = nn.Sequential(
            nn.Linear(k, 16), nn.Tanh(), nn.Linear(16, n_in))

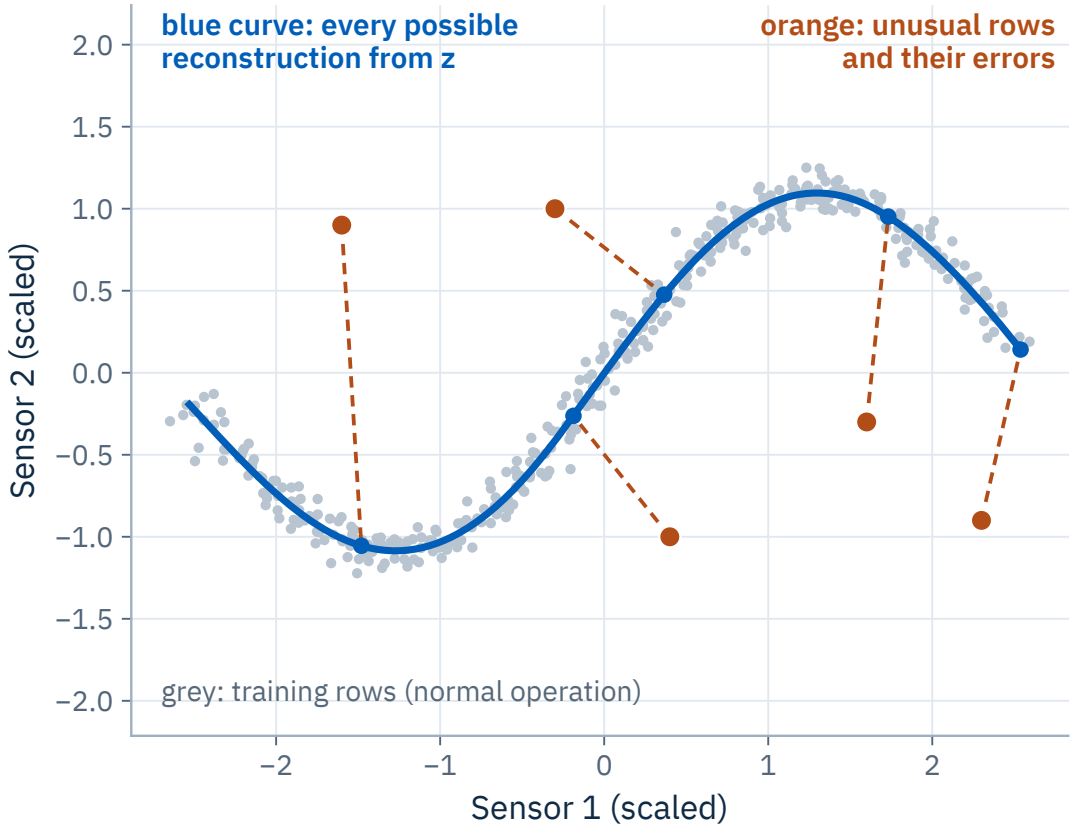
    def forward(self, x):
        return self.decoder(self.encoder(x))

model = AE()
loss = nn.functional.mse_loss(model(xb), xb)    # target = input
score = ((model(x) - x) ** 2).mean(dim=1)       # per-row error
```

Train on standardized rows from normal operation only; score new rows in `eval()` mode.

The bottleneck learns the shape of normal data

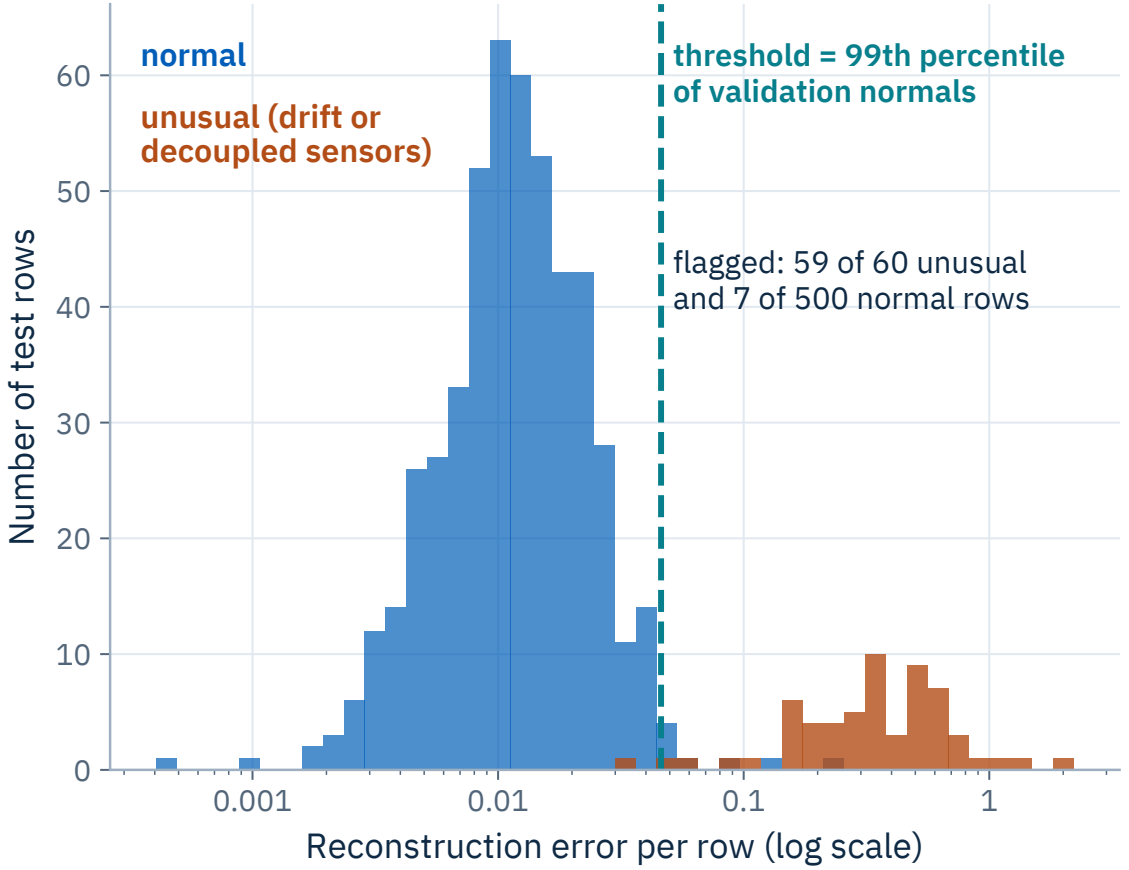
- With a one-number bottleneck, every output lies on one learned curve
- Normal rows sit near the curve and reconstruct well
- Unusual rows are pulled onto the curve: their error is large



Computed: 400 synthetic two-sensor rows near a curve (numpy seed 3); autoencoder 2-32-32-1-32-32-2 (tanh), torch seed 0; five hand-placed unusual rows.

Reconstruction error becomes an anomaly score

- Train on normal operation; score each row by its mean squared error
- Set the threshold on validation normals, for example their 99th percentile
- Here 59 of 60 unusual rows and 7 of 500 normal rows were flagged
- Evaluate events with precision-recall (ML Module 5)



Computed: synthetic 8-channel sensor rows driven by two hidden factors (numpy seed 11); autoencoder 8-16-2-16-8 trained on 2,000 normal rows, torch seed 0.

Reconstruction scores need careful interpretation

Threshold choice

- A percentile sets the alarm rate, not accuracy
- Choose it on validation, report on test

Too much capacity

- A wide bottleneck can reconstruct anomalies too
- Compare with a PCA reconstruction baseline

Contaminated training

- Unlabelled faults in "normal" data raise the bar
- Audit the training window first

Where next

- ML Module 4: autoencoders versus PCA
- Days 4 and 5: convolutional and sequence versions

LABORATORY

55 min

Laboratory 2: regularization or autoencoder

- Track A: compare baseline, dropout, batch-norm, and weight-decay MLPs
- Track A: hold out 10,000 training images for validation; test once
- Track B (04_nn_unsupervised): autoencoder with a one-number bottleneck
- Track B: set the threshold on validation rows, then inspect flagged points
- Both: change one thing at a time and record the budget and seeds

NOTEBOOK

`exercises/03_advanced_nn/Advanced_NN_MNIST_Student.ipynb`

EVIDENCE TO SUBMIT

- A: table of validation accuracy and train-validation gap
- B: error histogram, threshold, flagged points
- One sentence on what the result cannot show

Track A needs torchvision and MNIST downloaded before class; Track B runs offline.

DISCUSSION

Lab 2 debrief: evidence over impressions

Track A

- Was validation separate from test?
- Is the gap larger than seed-to-seed noise?

Track B

- Was the threshold fitted on validation rows?
- Where do the flagged points lie, and why?

Both tracks

- Budget and seeds reported
- Claims limited to this data

◆ Every claim carries its budget, its seeds, and its limits.

Key takeaways

- ✓ Diagnose in a fixed order: data, initial loss, one batch, gradients, validation.
- ✓ Initialization, normalization, and skip paths keep signals at a trainable scale.
- ✓ `eval()` and no-grad do different jobs; copy the best checkpoint and verify a reload.
- ✓ Learning rate first: range test, log-scale search, and a schedule on a declared clock.
- ✓ Regularize for the symptom you see, and validate every change over several seeds.
- ✓ Reconstruction error is a score; its threshold is a decision made on validation data.

PRACTICE

10 min · Individual

Exit check

1. What is the difference between `model.eval()` and `torch.no_grad()`?
2. Why does early stopping use validation rather than test data?
3. Why can adding L2 to Adam differ from AdamW weight decay?
4. Your initial loss on a balanced 5-class problem is 9.8. What do you check first?
5. Name one augmentation that would be invalid for your own sensor data, and why.

References and further study

- Goodfellow, Bengio, and Courville (2016), *Deep Learning*, chapters 7, 8, and 11; Karpathy (2019), "A recipe for training neural networks".
- Glorot and Bengio (2010) on initialization; He et al. (2015) on rectifier initialization; He et al. (2016), deep residual learning.
- Ioffe and Szegedy (2015), batch normalization; Ba, Kiros, and Hinton (2016), layer normalization; Wu and He (2018), group normalization.
- Kingma and Ba (2015), Adam; Loshchilov and Hutter (2019), AdamW; Smith (2017), cyclical learning rates; Srivastava et al. (2014), dropout.
- Bergstra and Bengio (2012), random search; Belkin et al. (2019), double descent; Hinton and Salakhutdinov (2006), autoencoders; Google Research, *Deep Learning Tuning Playbook*; PyTorch documentation.